Raport projekt CNN - klasyfikacja postaci z filmu "Madagaskar"

1. Baza danych obrazów użyta do klasyfikacji

Podczas tworzenia projektu korzystałem z własnoręcznie stworzonego zbioru danych postaci z filmu animowanego "Madagaskar". Składa się on z 250 obrazów w formacie PNG rozdzielonych na 4 klasy. Zbiór został podzielony w proporcjach:

- 200 - zbiór treningowy
- 50 - zbiór walidacyjny

Zbiór składa się z nastepujących klas:

- Alex - 55 obrazów
- Gloria - 66 obrazów
- Marty - 68 obrazów
- Melman - 61 obrazów

Przykładowe dane ze zbioru:



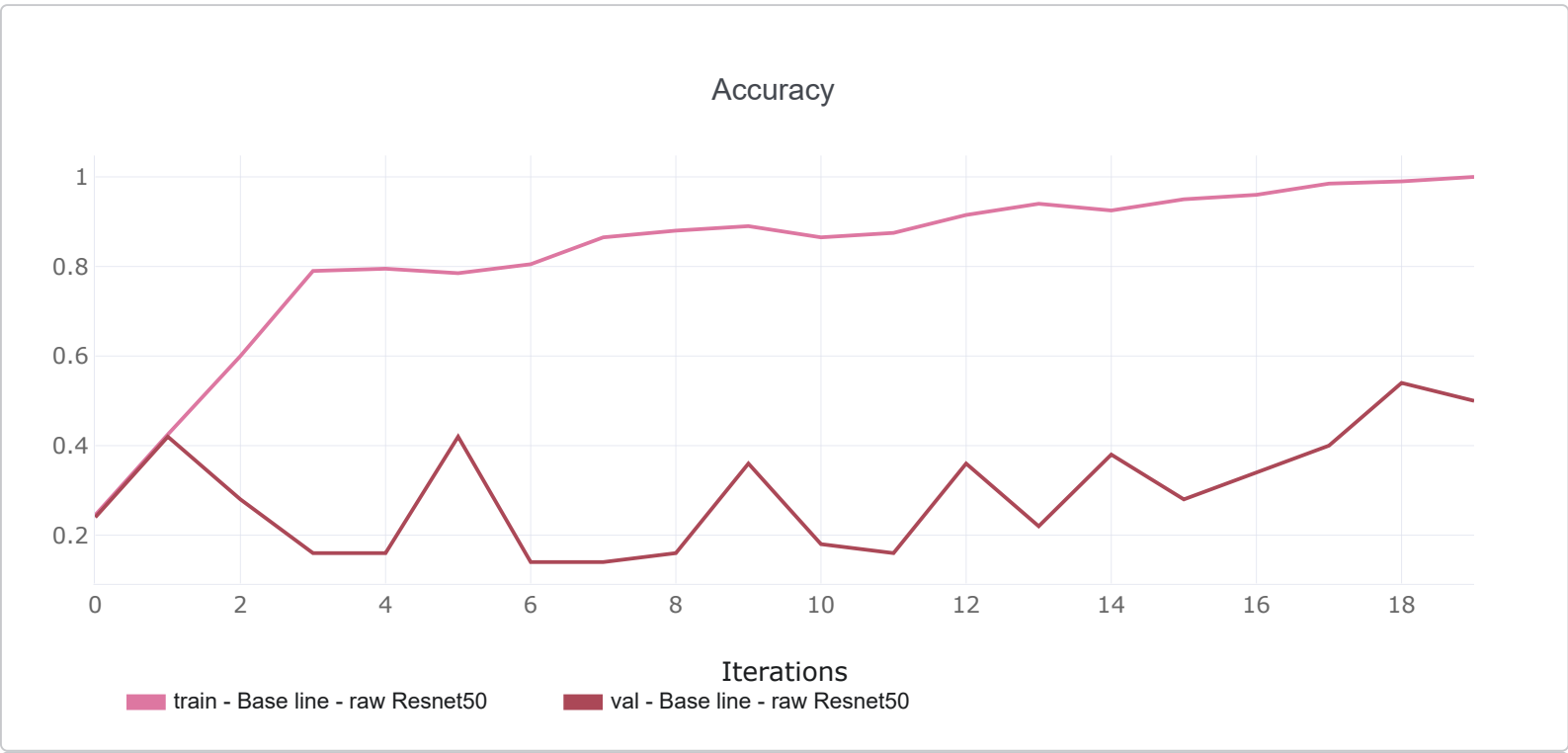


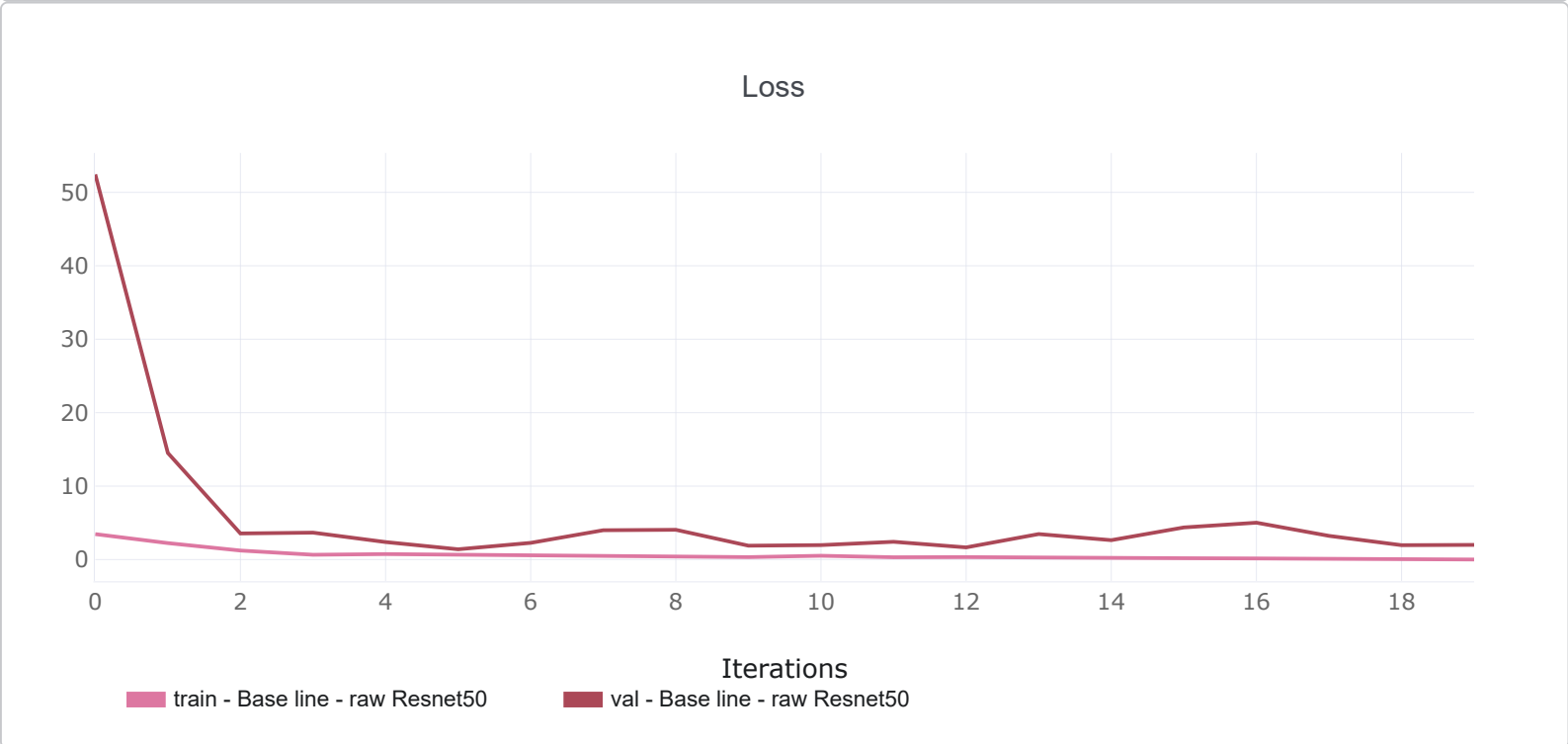

2. Model ResNet50 wytrenowany od zera na CPU

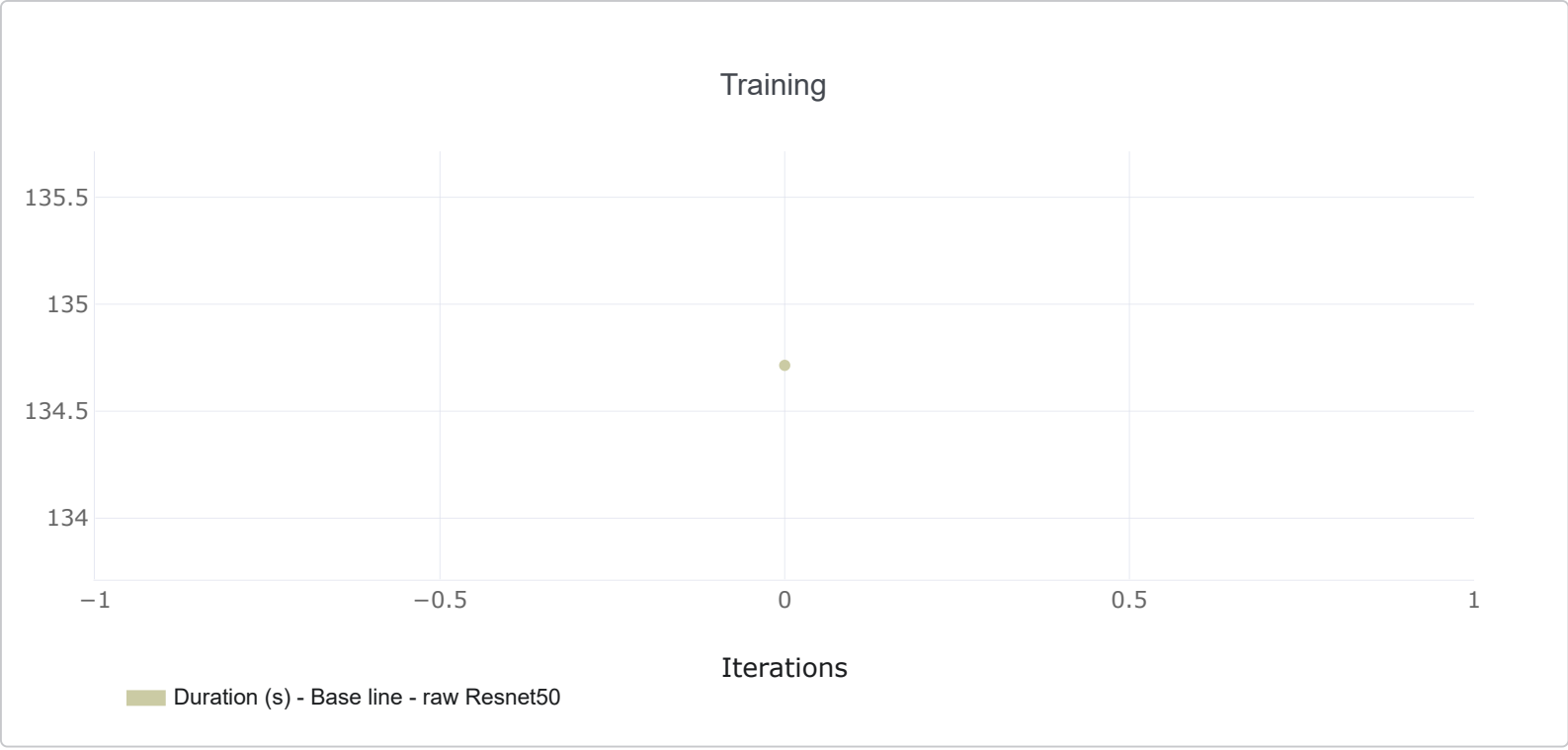
Parametry modelu:

```
augmentation: False
batch_size: 16
dropout: 0
epochs: 20
extra_data: False
img_shape: [64, 64]
```

```
model_name: resnet50
normalization: False
transfer_learning: False
```







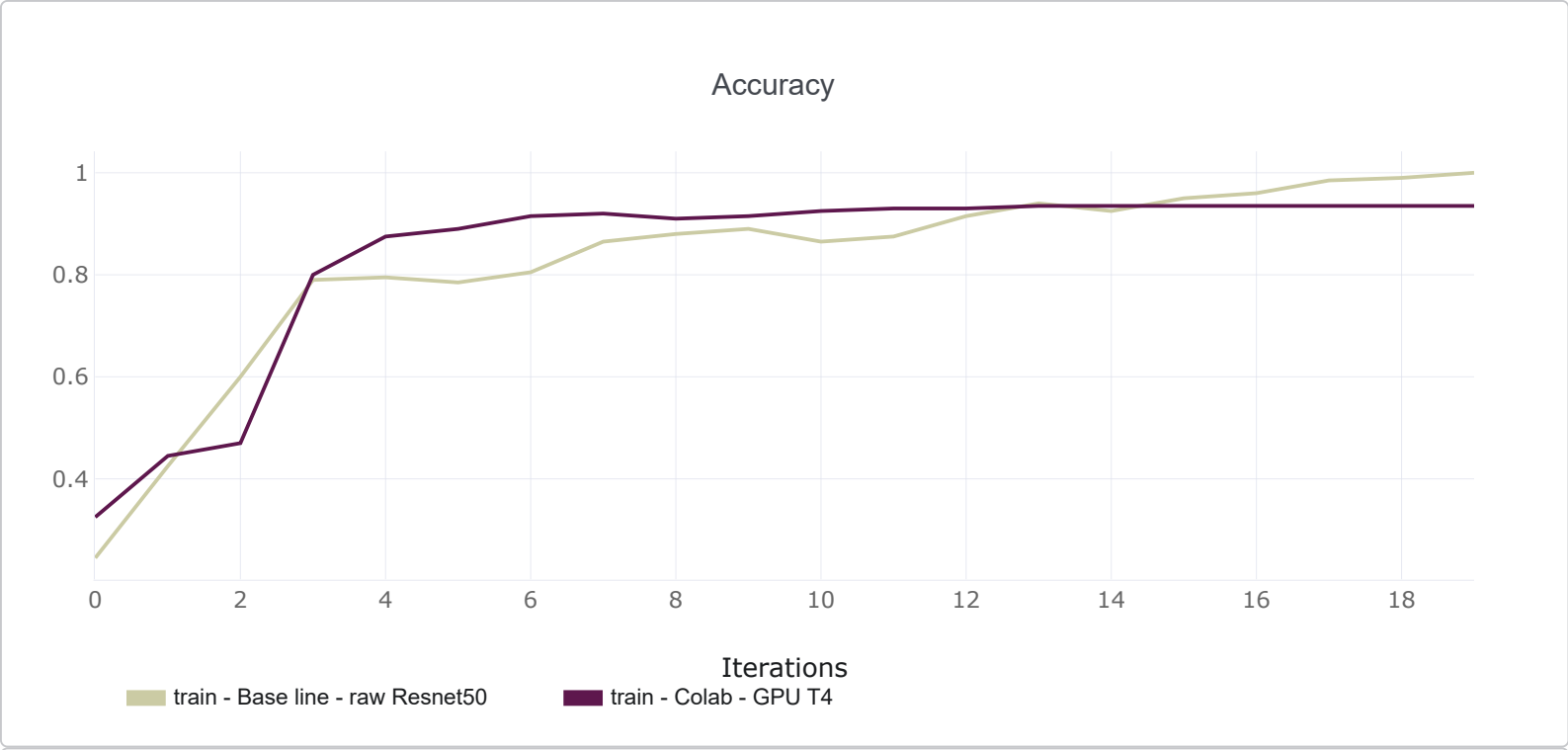
Jak widać na powyższych skalarach trenowanie modelu trwało 134 sekundy ze średnią dokładnością na poziomie 34% na zbiorze walidacyjnym.

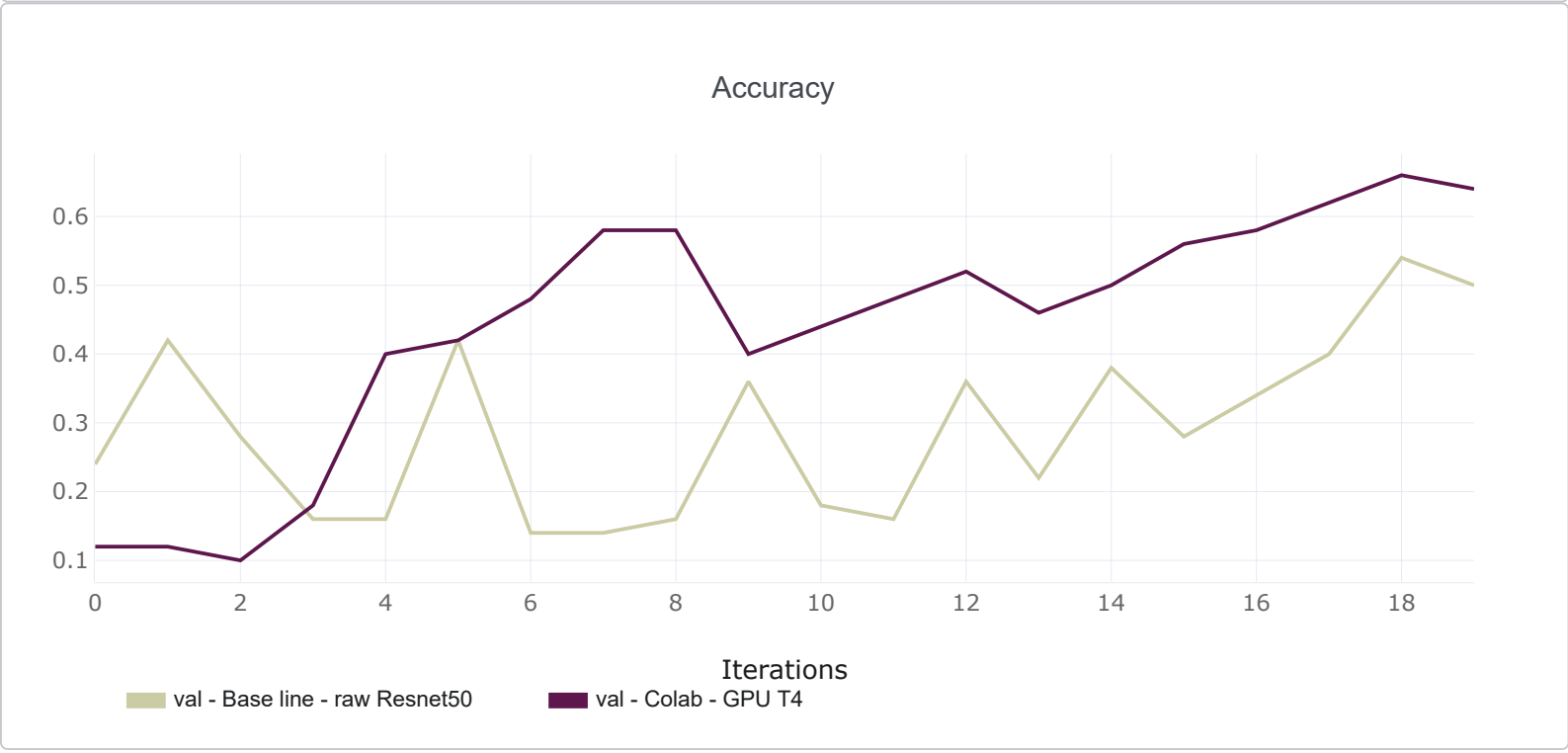
3. Opytmalizacja szybkości treningu

3a. Akcelerator GPU

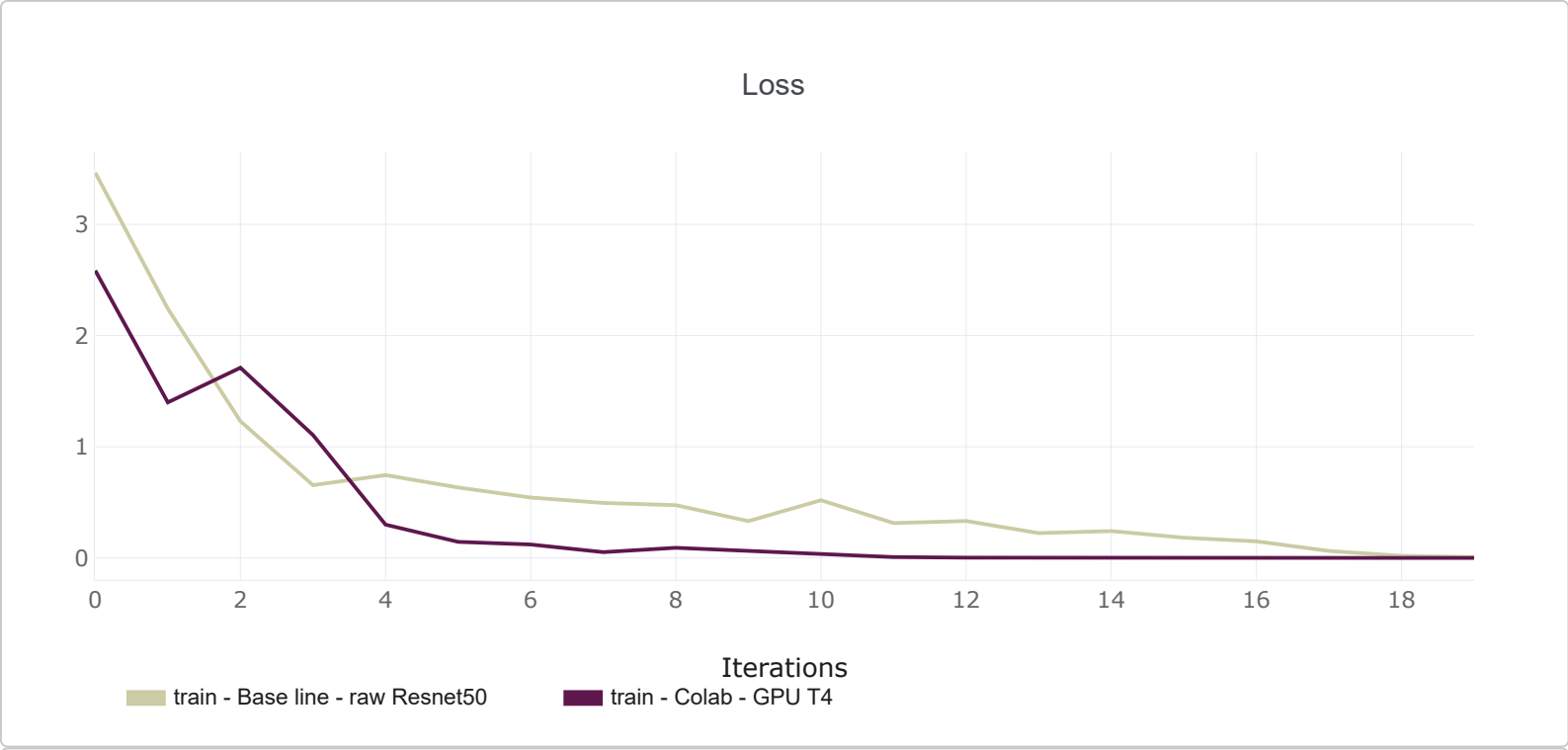
Uczenie przeprowadzone na Colab - GPU T4. Próbowałem zrobić to na własnym sprzęcie, ale dużo problemów było w tensorflow z obsługą GPU - ostatnia wersja wspierająca GPU to 2.10 i było ciężko zgrać żeby wszystkie sterowniki były ze sobą kompatybilne. Na dodatek moja karta mimo, że NVIDIA to akurat seria 1600 nie obsługuje CUDA.

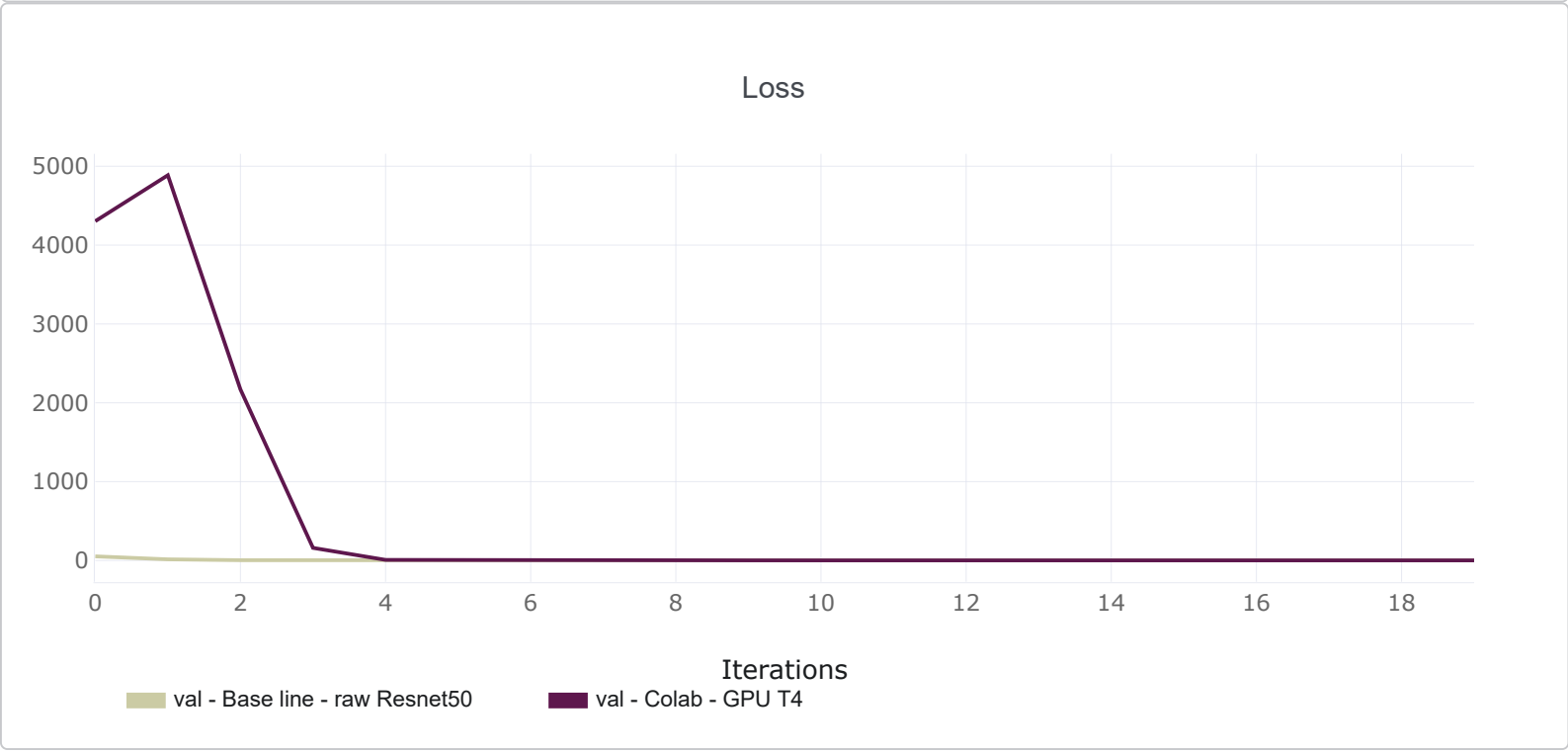
Accuracy


Accuracy
1
0.8
0.6
0.4
0
2
4
6
8
10
12
14
16
18
Iterations
train - Base line - raw Resnet50
train - Colab - GPU T4


Accuracy
0.6
0.5
0.4
0.3
0.2
0.1
0
2
4
6
8
10
12
14
16
18
Iterations
val - Base line - raw Resnet50
val - Colab - GPU T4

Loss



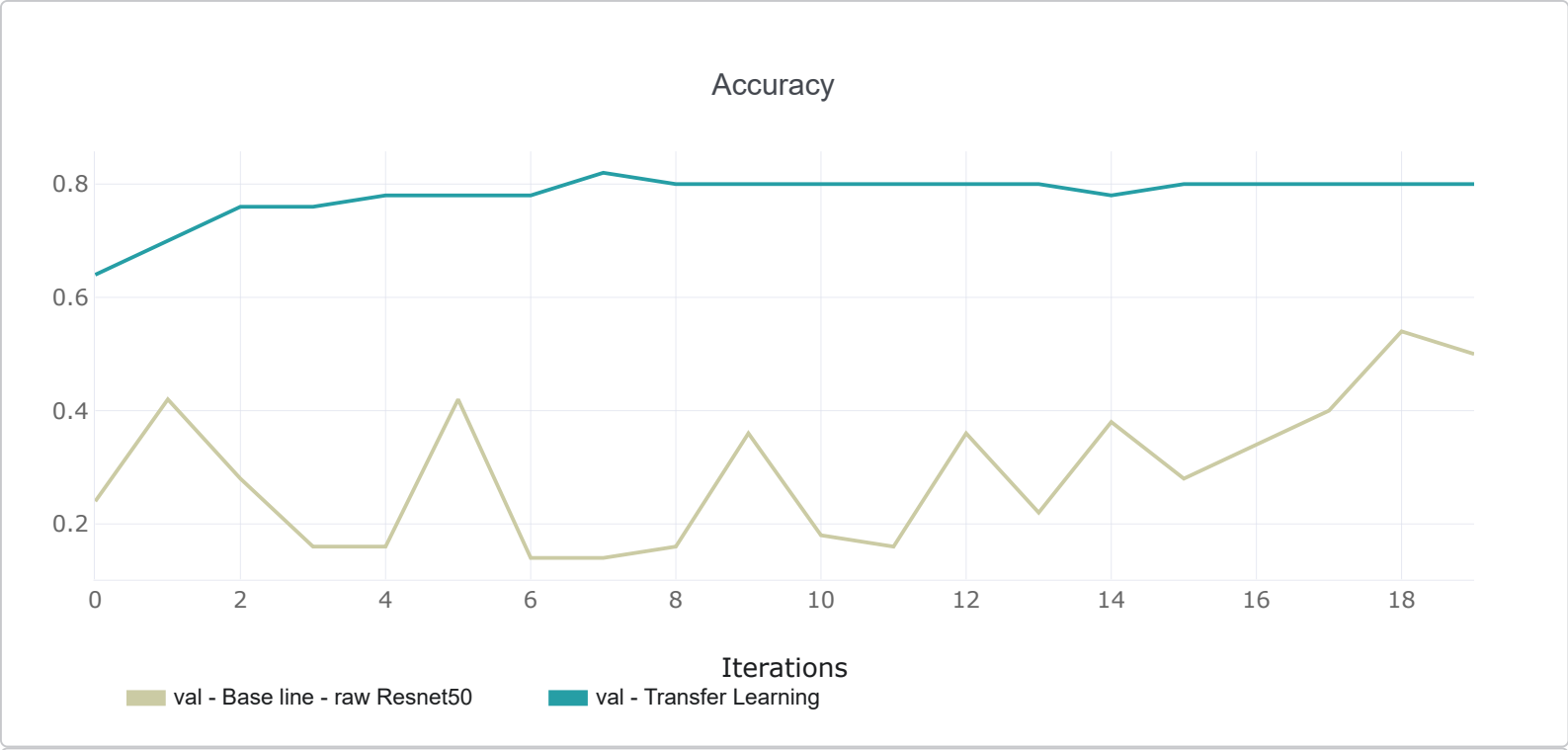


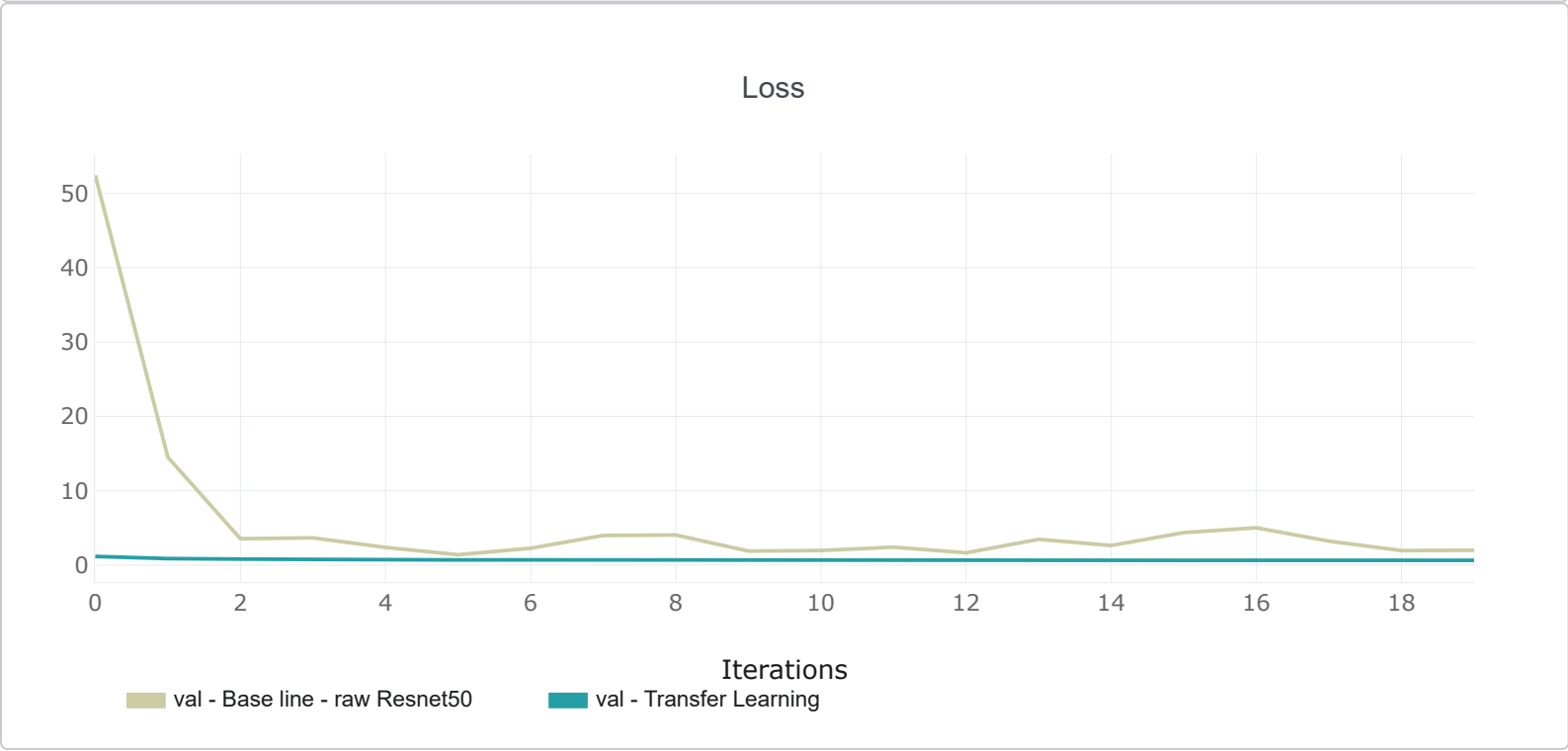
Training time

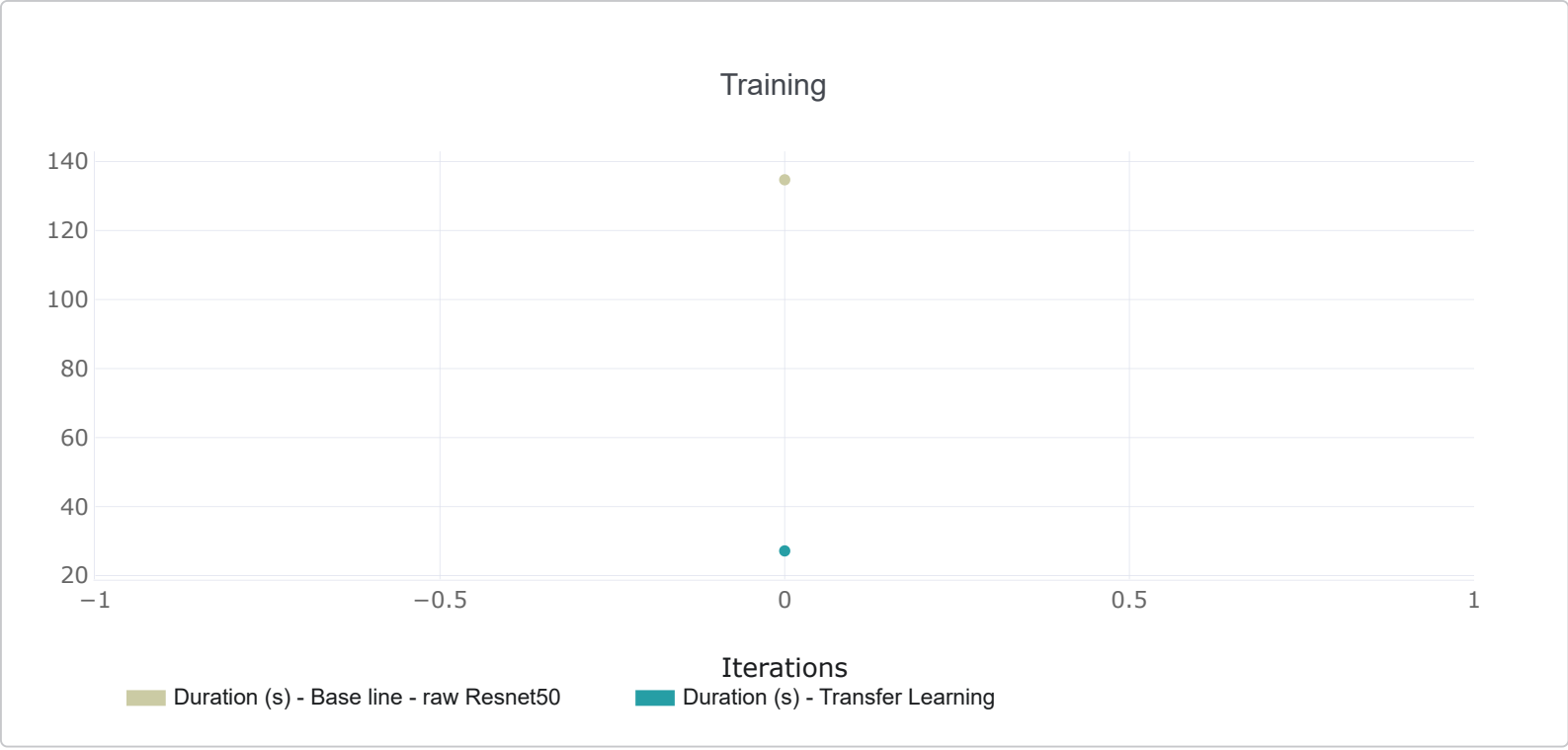


Po przeprowadzeniu uczenia przy użyciu GPU możemy zauważyć poprawę dokładności z 34% na 64%. Można róznież zaobserwować wygładzenie się wykresów dokładności. Czas uczenia nieznacznie uległ poprawie - zmniejszył się o ok. 6 sekund.

3b. Transfer learning


Accuracy
0.8
0.6
0.4
0.2
0
2
4
6
8
10
12
14
16
18
Iterations
val - Base line - raw Resnet50
val - Transfer Learning


Loss
50
40
30
20
10
0
0
2
4
6
8
10
12
14
16
18
Iterations
val - Base line - raw Resnet50
val - Transfer Learning



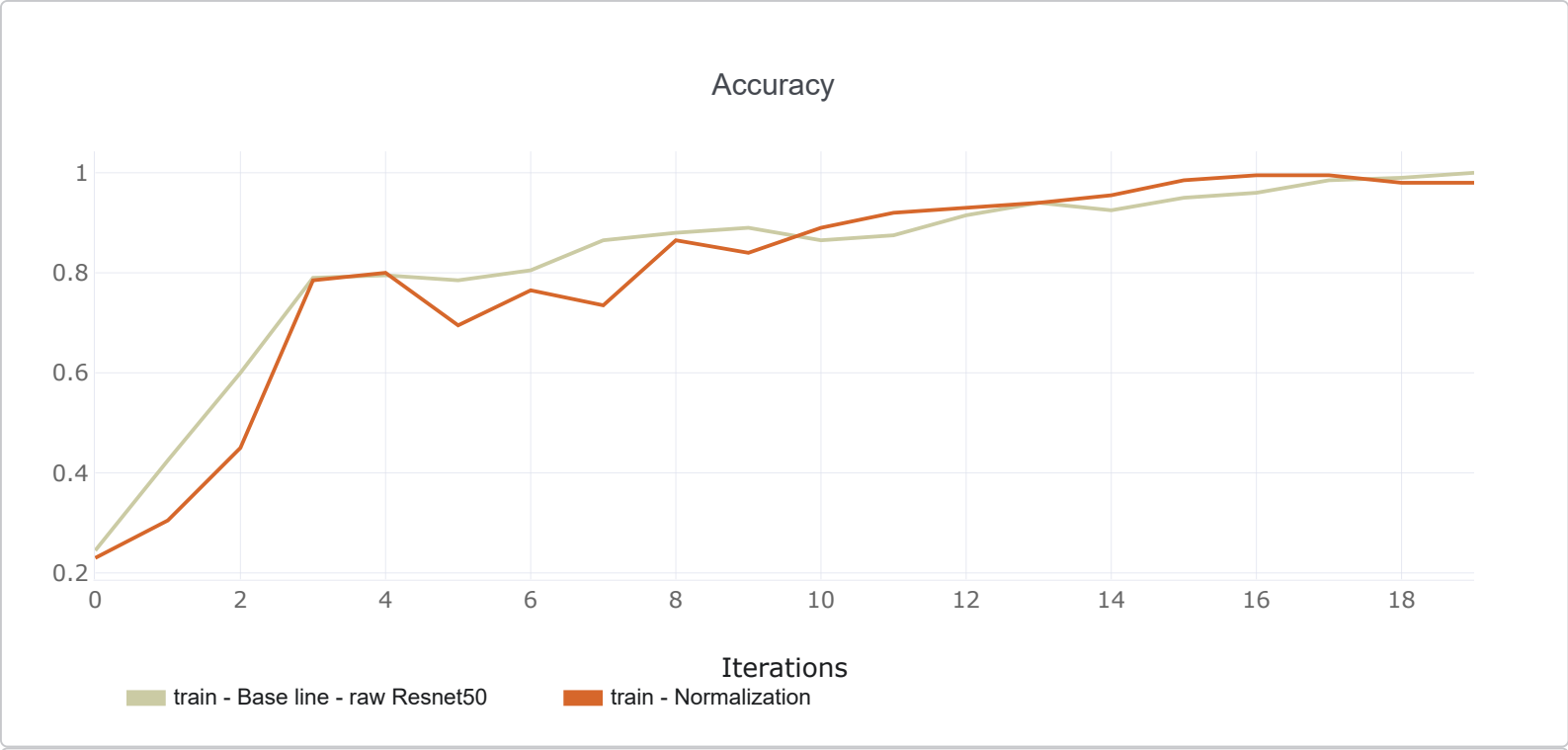
Odnotowana została znacząca poprawa poziomu dokładności na 80% z bazowego poziomu równego 34%. Poprawie uległ też czas treningu , który udało się zmniejszyć ze 134 sekund na 27.

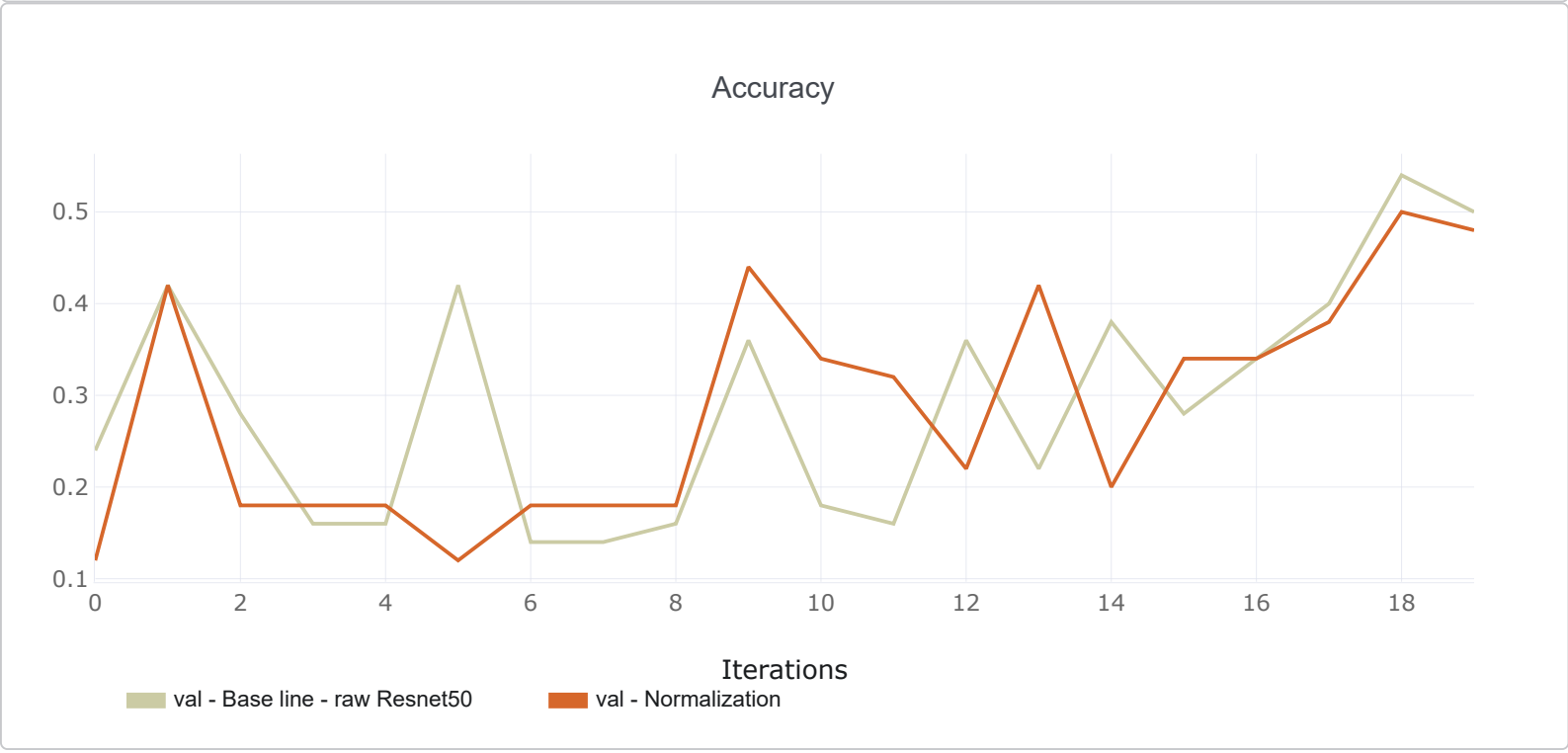
4. Optymalizacja dokładności

4.a - Normalizacja

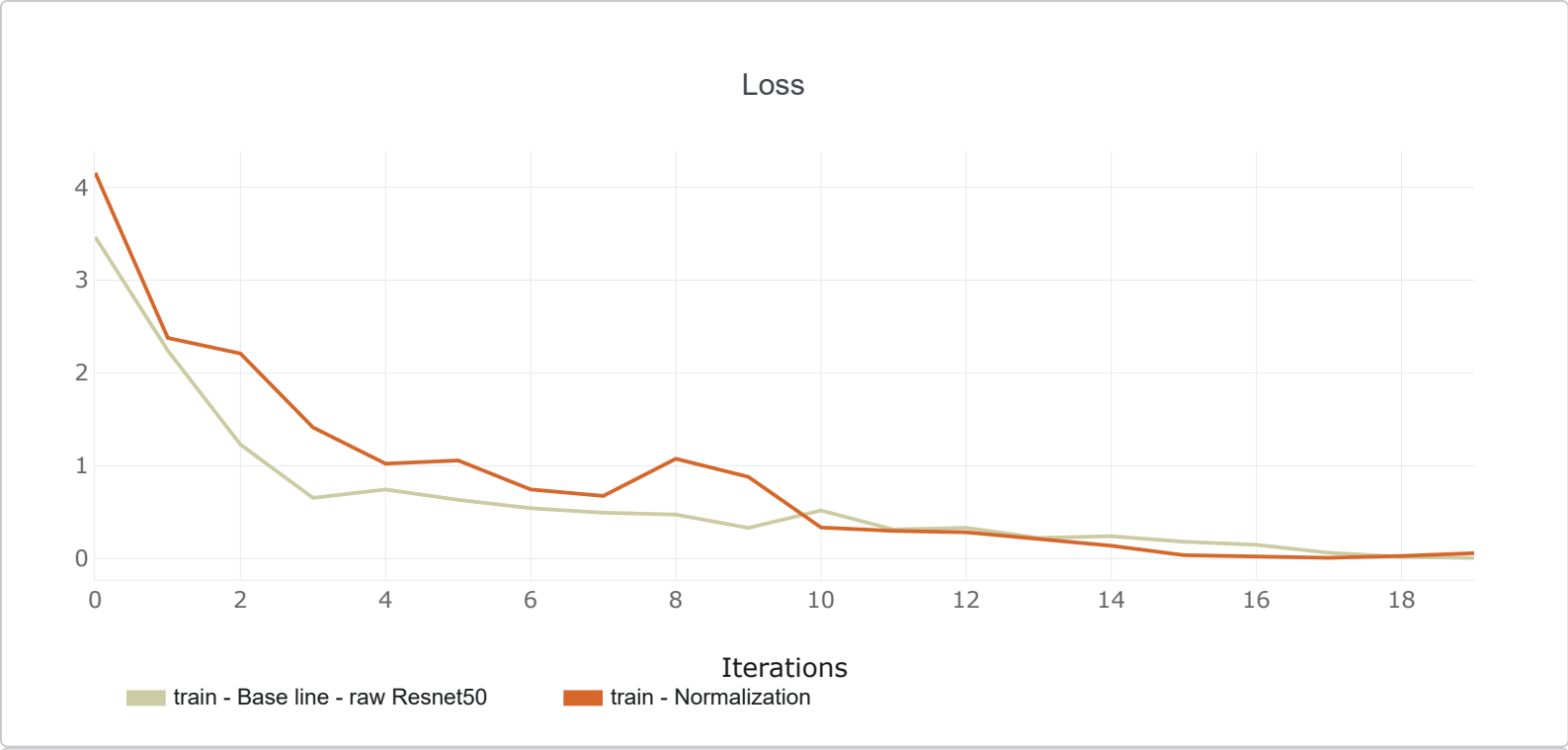
```
normalization_layer = tf.keras.layers.Rescaling(1. / 255)
ds = ds.map(lambda x, y: (normalization_layer(x), y))
```

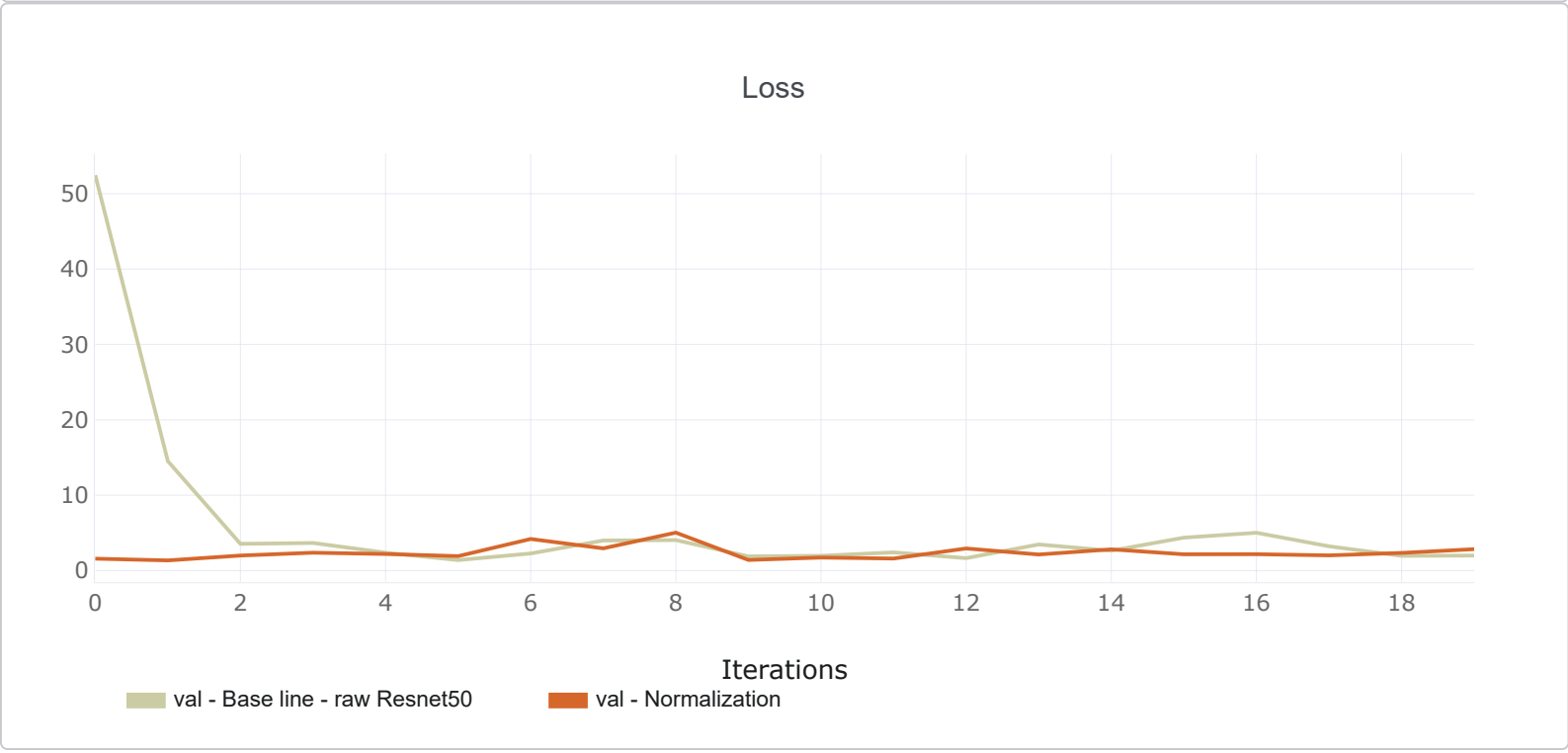
Accuracy


Accuracy
1
0.8
0.6
0.4
0.2
0
2
4
6
8
10
12
14
16
18
Iterations
train - Base line - raw Resnet50
train - Normalization

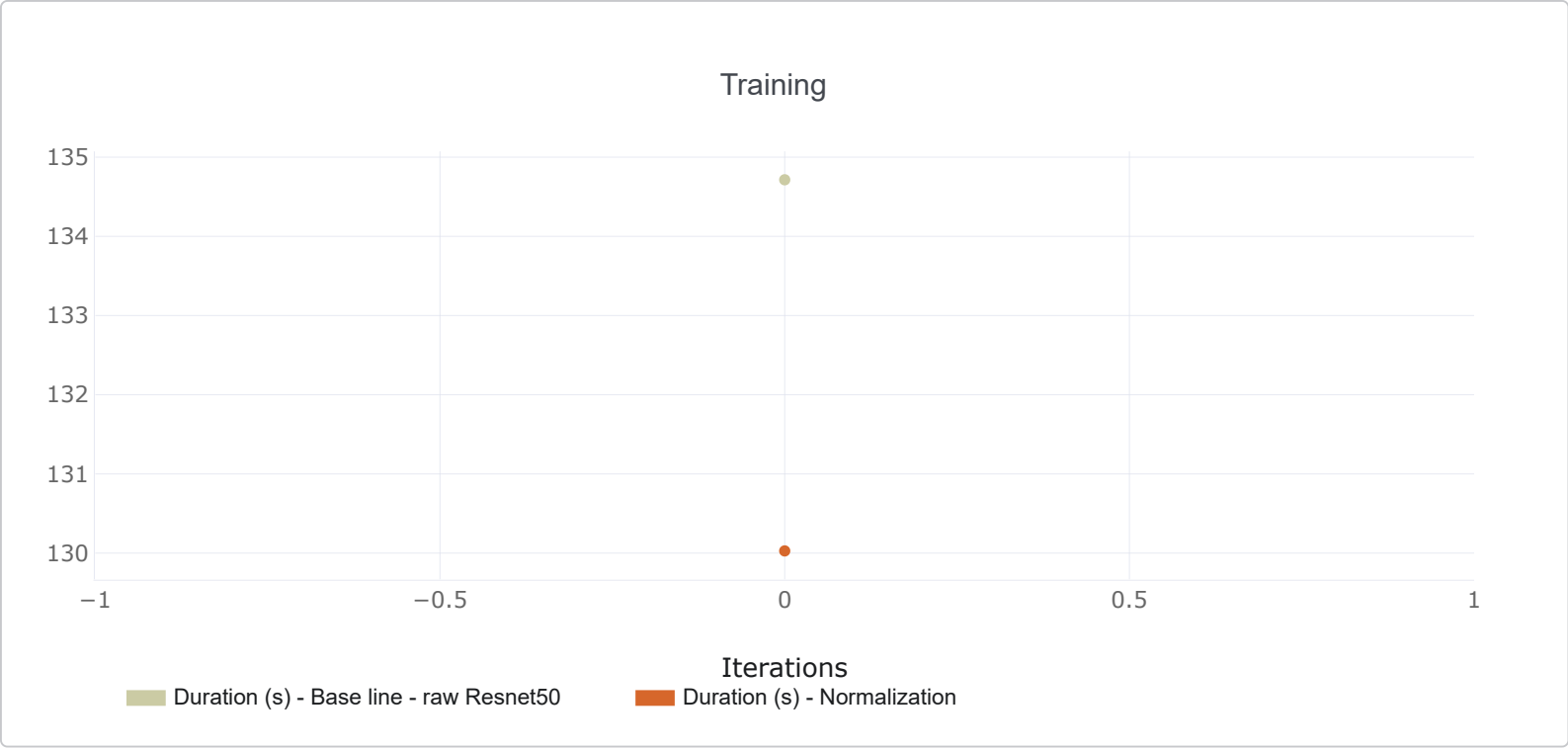

Accuracy
0.5
0.4
0.3
0.2
0.1
0
2
4
6
8
10
12
14
16
18
Iterations
val - Base line - raw Resnet50
val - Normalization

Loss





Training time

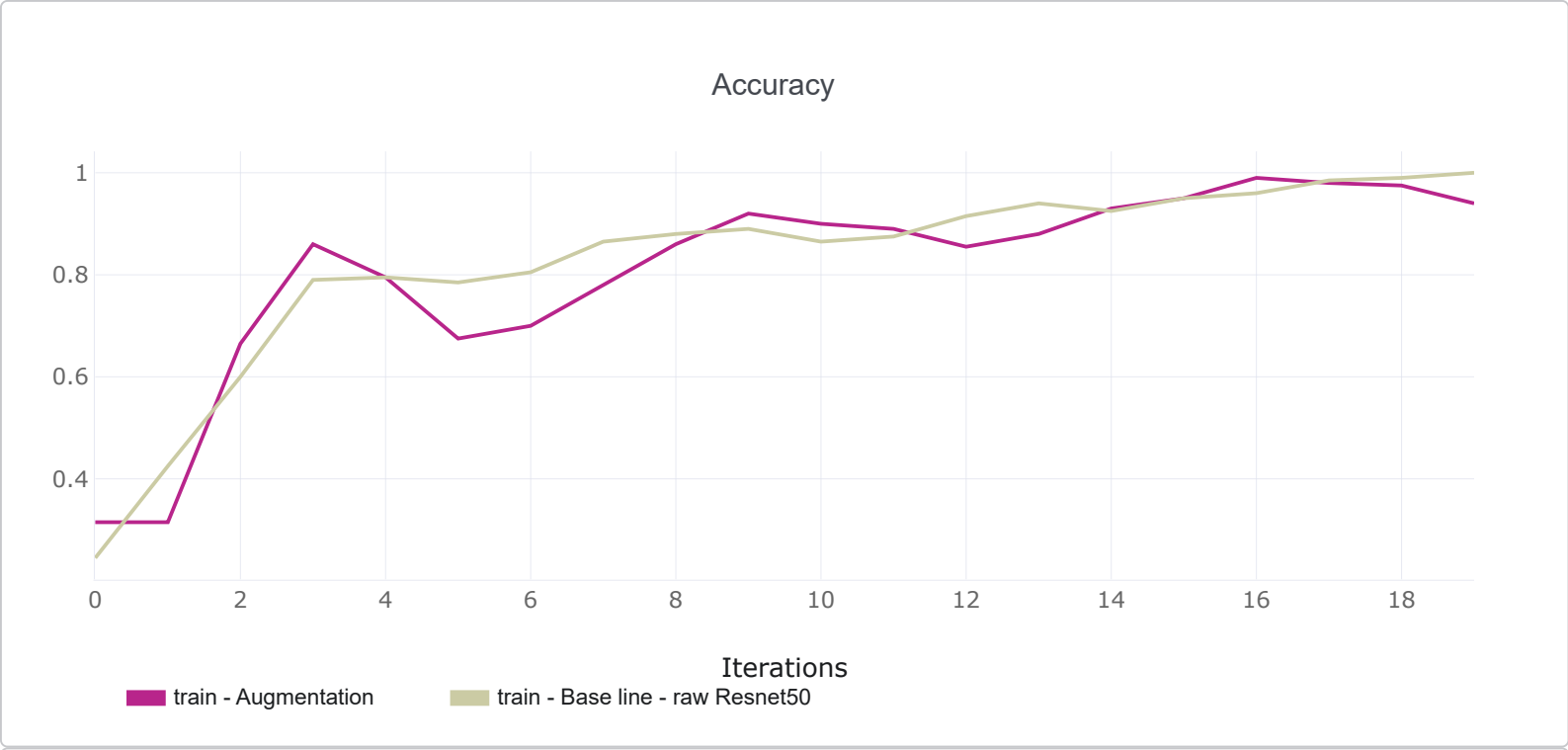


Nie odnotowano dużego wpływu na wykres przebiegu uczenia. Dokładnośc poprawiła się o 14 pp, a czas zmniejszył się o 4 sekundy.

4.b - Augmentacja

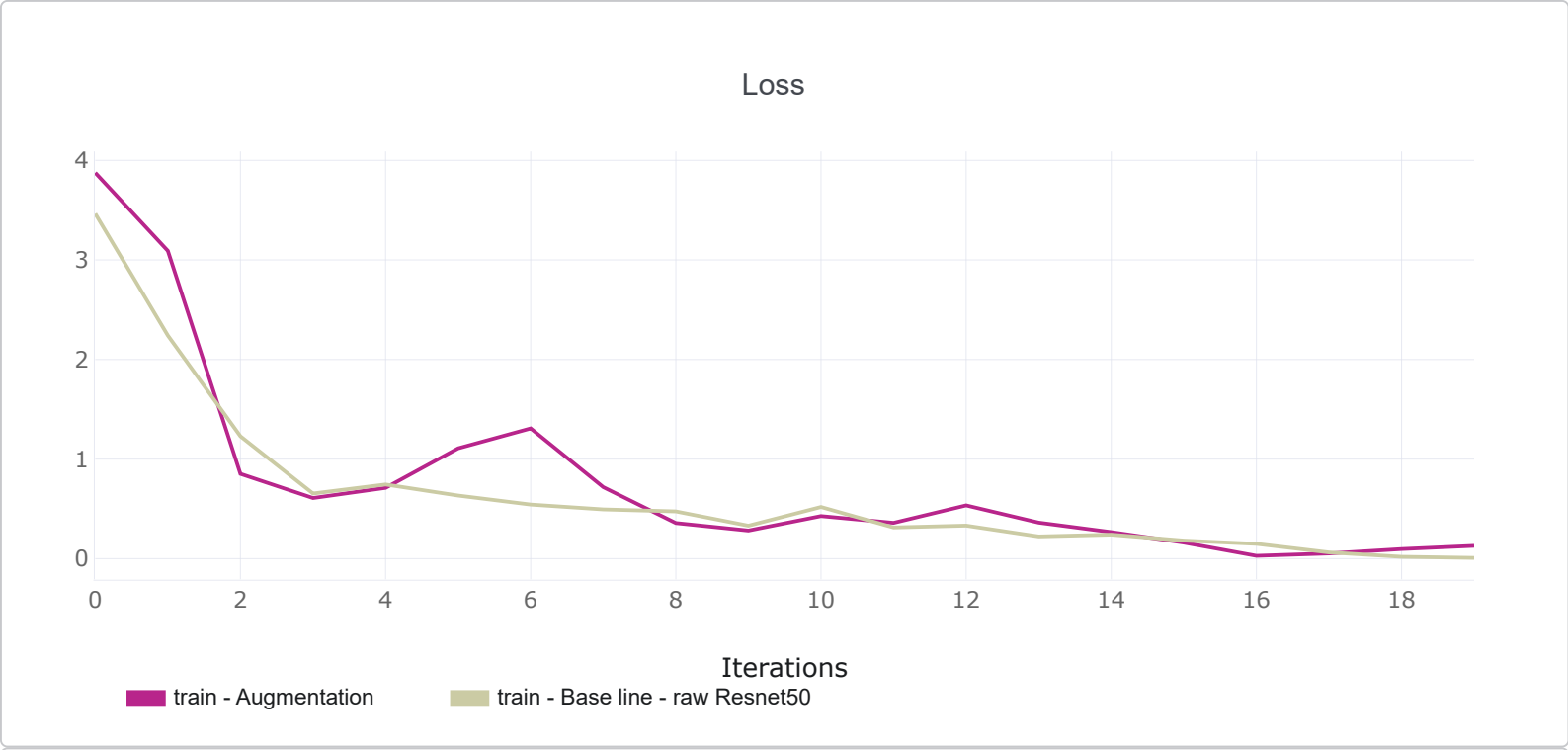
```
augmentation_layers = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])
ds = ds.map(lambda x, y: (augmentation_layers(x, training=True), y))
```

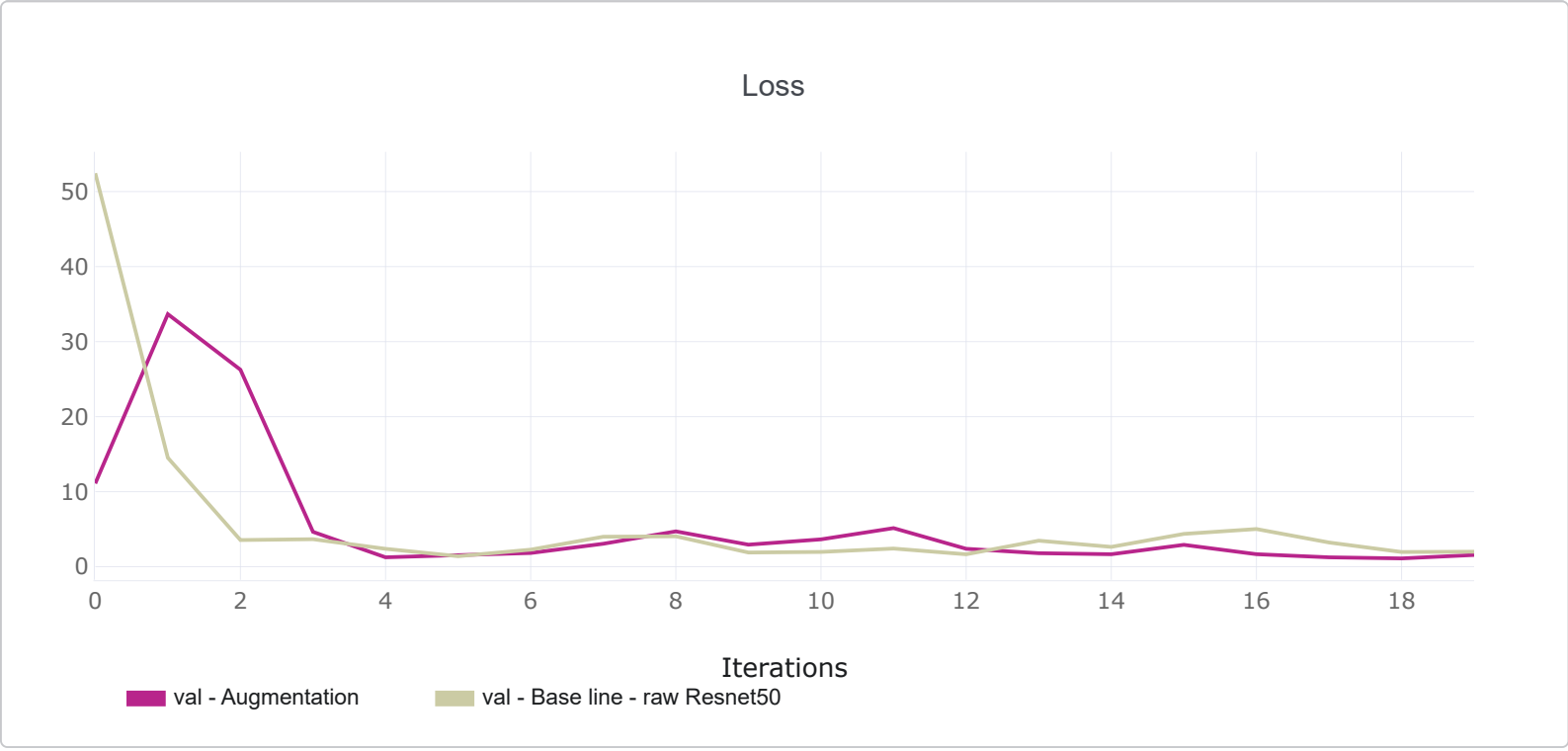
Accuracy


Accuracy
1
0.8
0.6
0.4
0
2
4
6
8
10
12
14
16
18
Iterations
train - Augmentation
train - Base line - raw Resnet50

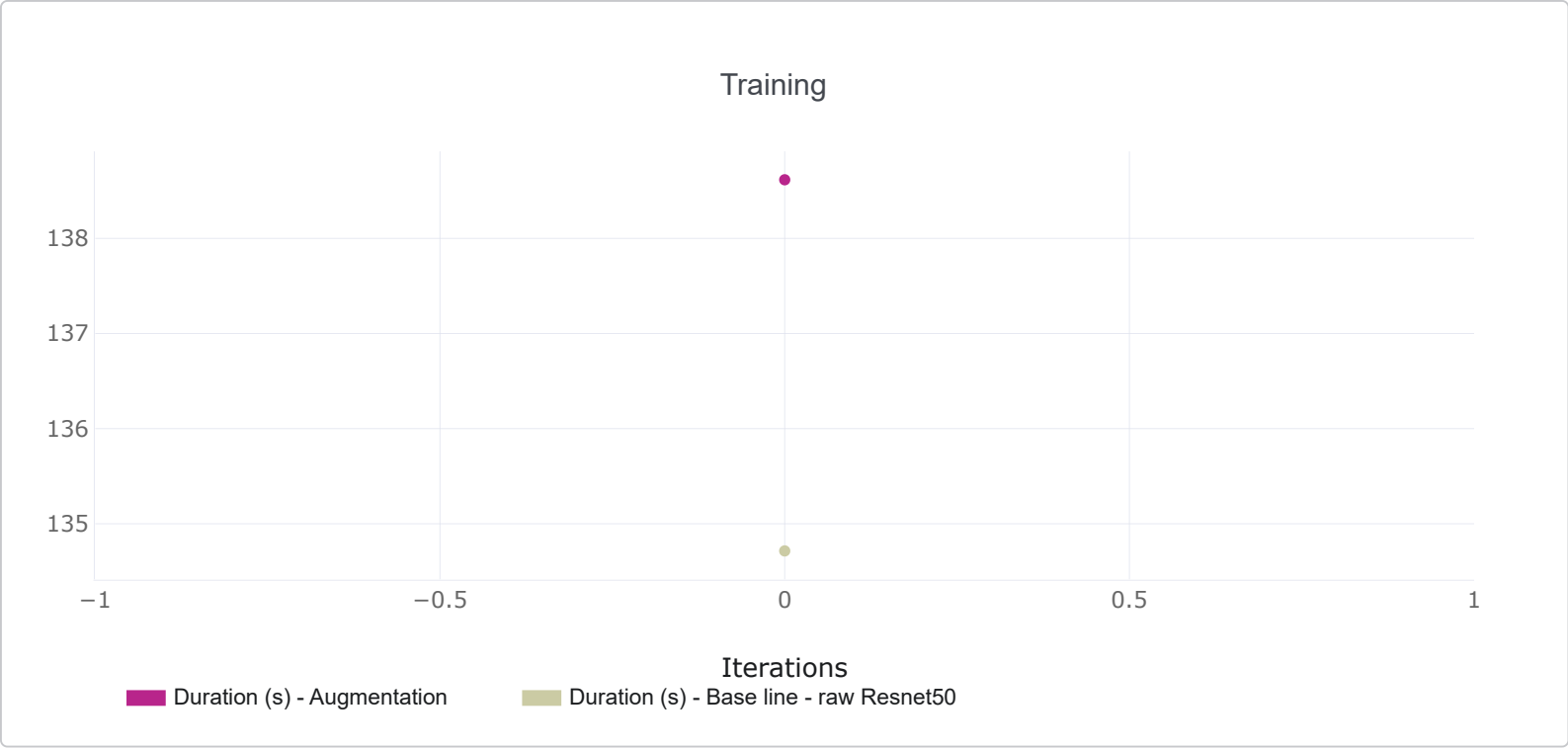

Accuracy
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
2
4
6
8
10
12
14
16
18
Iterations
val - Augmentation
val - Base line - raw Resnet50

Loss





Training time

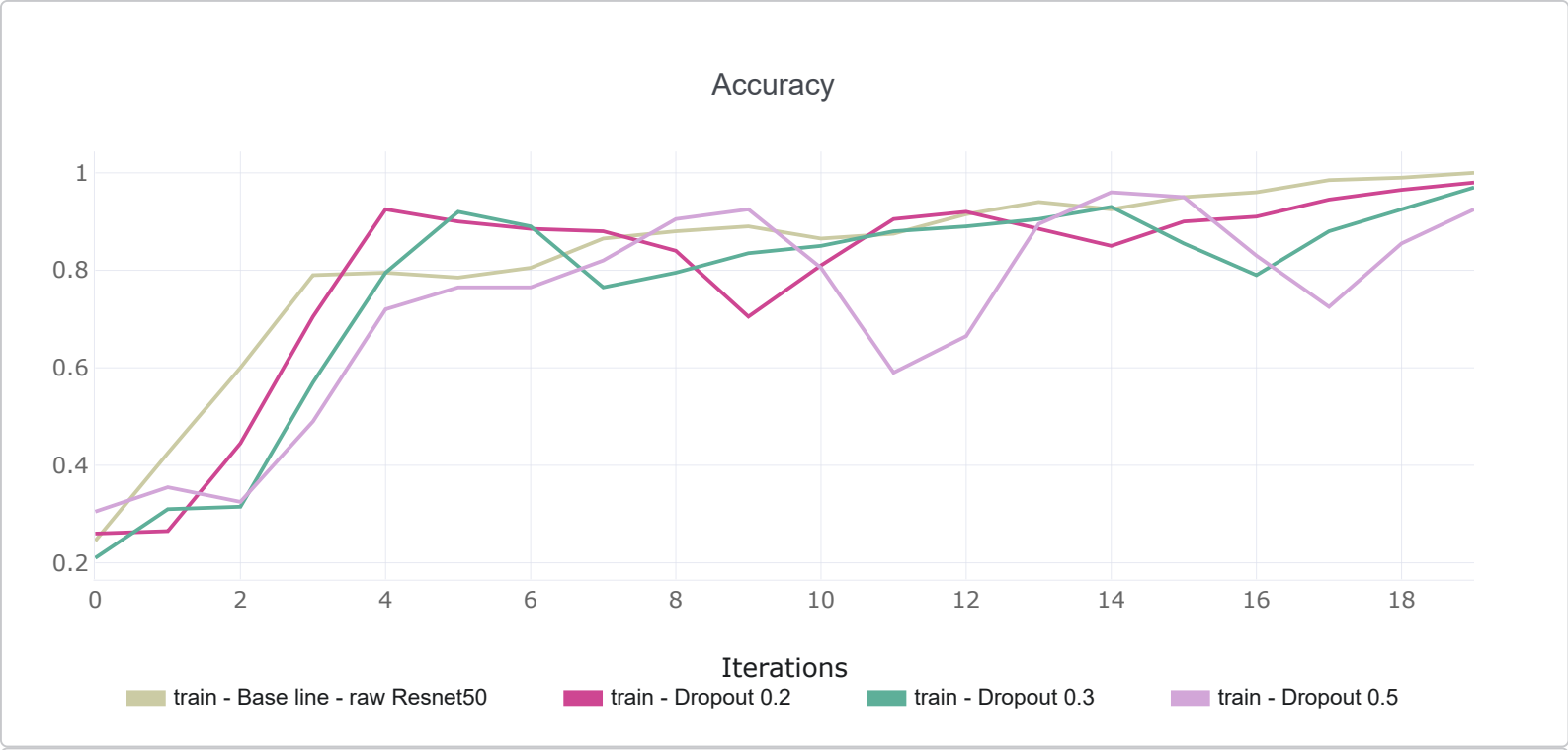


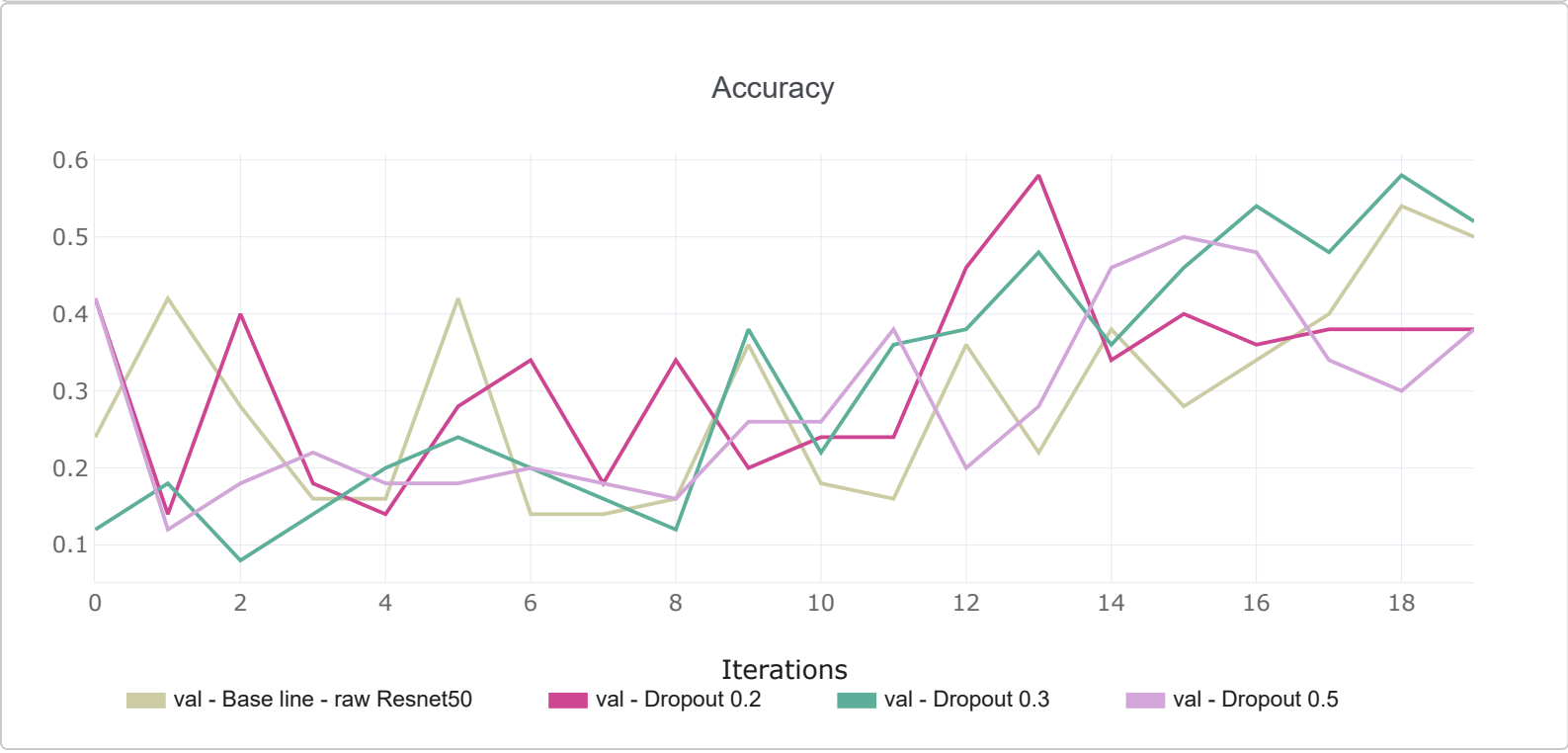
Zastosowanie augmentacji danych poprawiło wyniki o ok. 30 pp względem baseline'u. Czas trenowania wydłużył się nieznacznie bo o ok. 4 sekundy.

4.c - Dropout

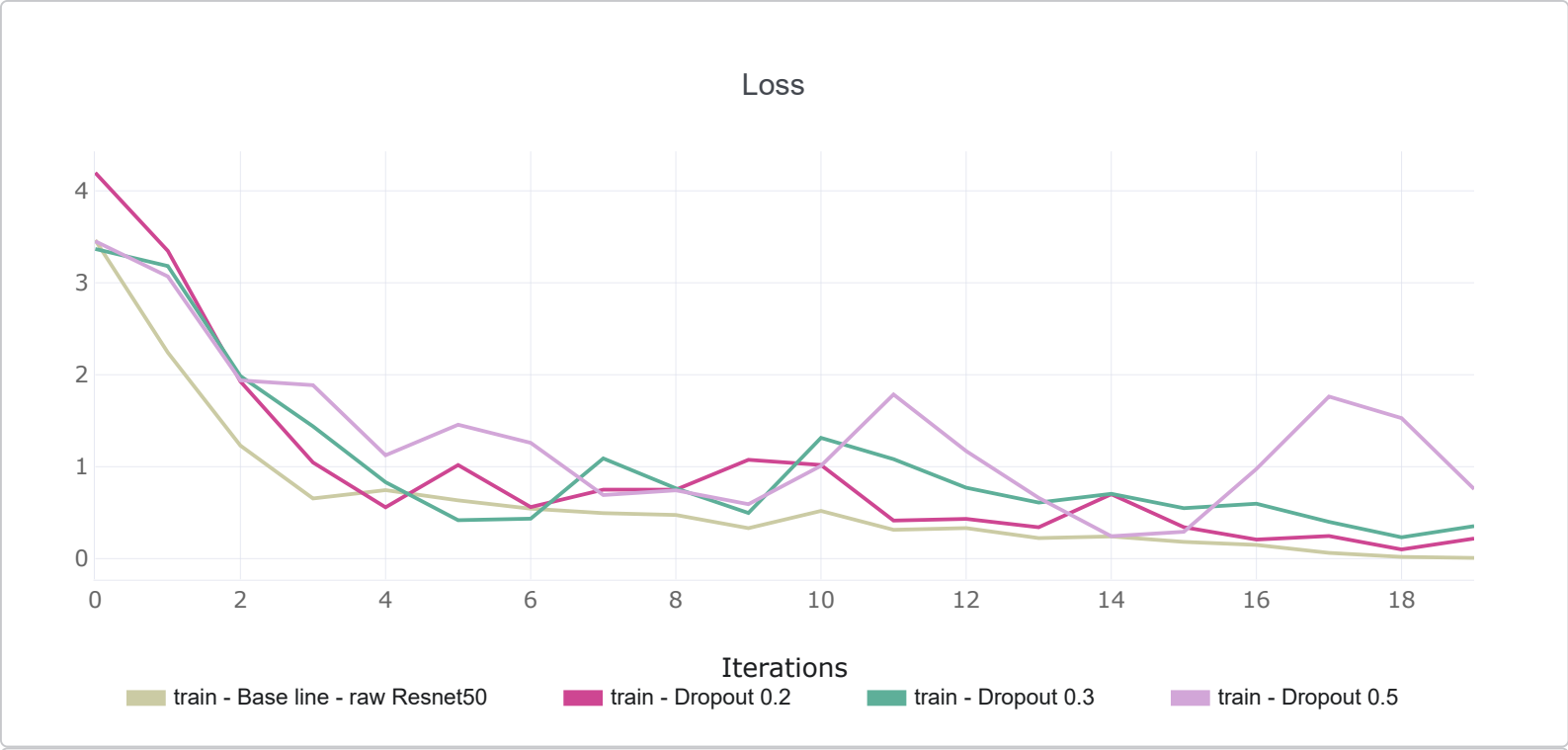
```
x = layers.Dropout(params["dropout"])(x)
```

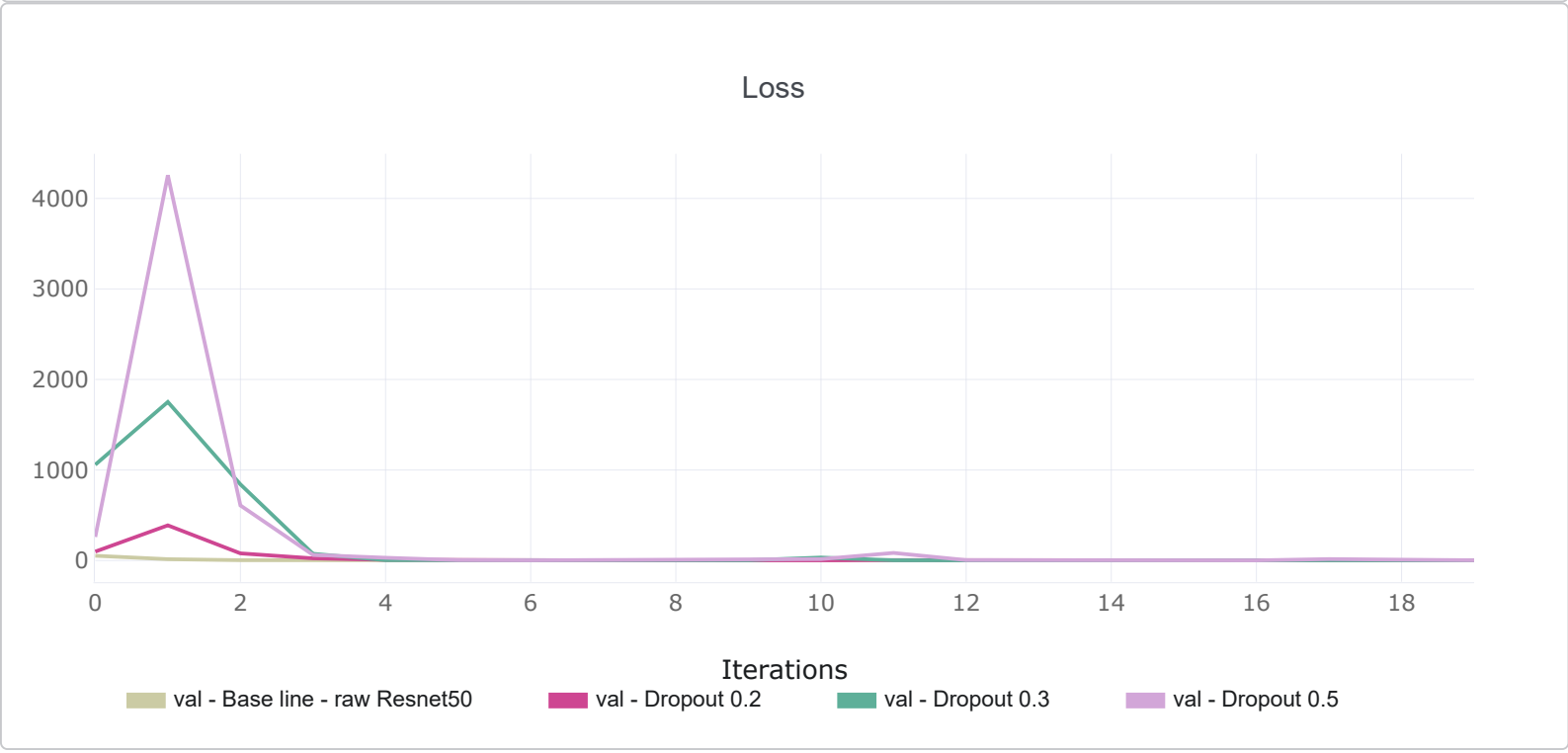
Accuracy


Accuracy
1
0.8
0.6
0.4
0.2
0
2
4
6
8
10
12
14
16
18
Iterations
train - Base line - raw Resnet50
train - Dropout 0.2
train - Dropout 0.3
train - Dropout 0.5

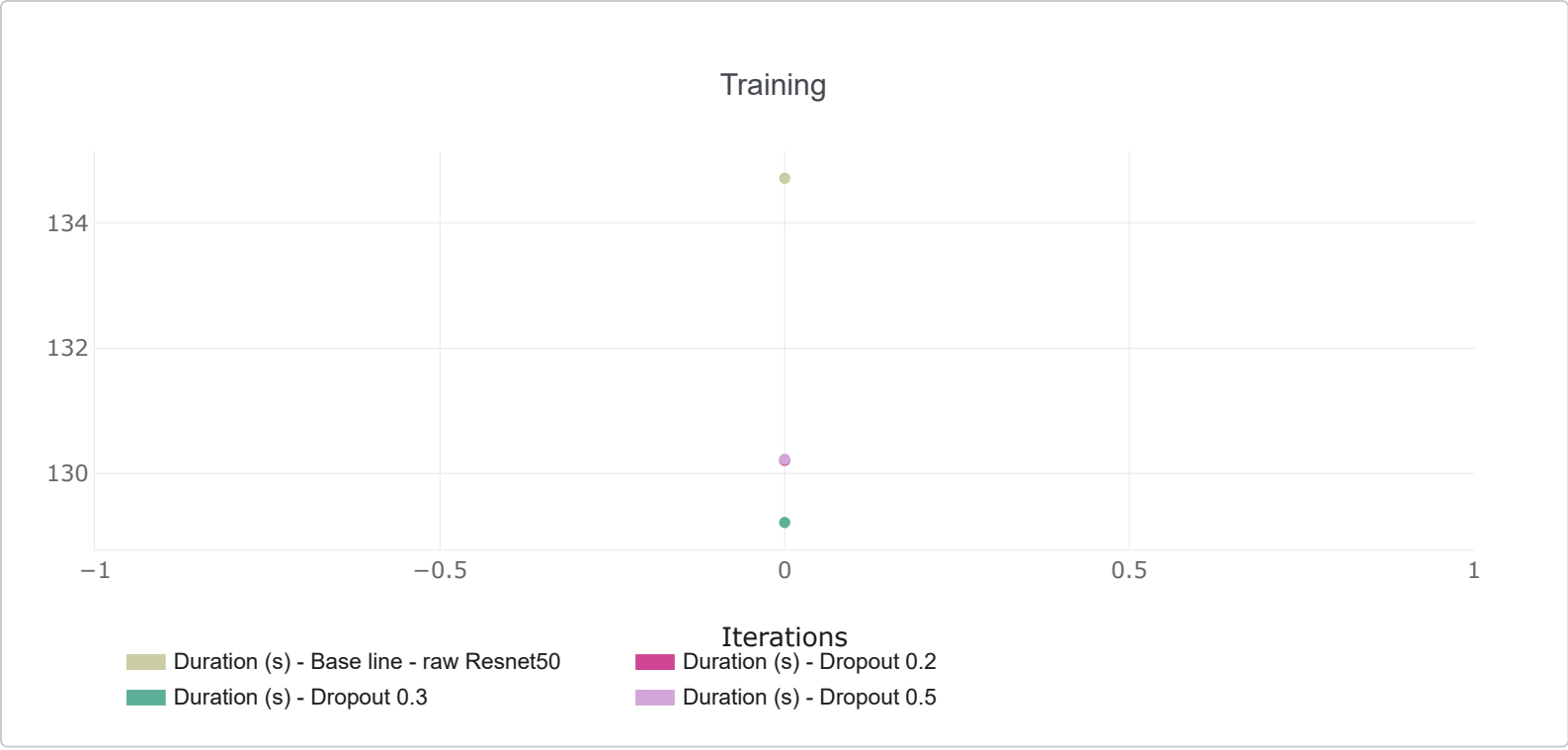

Accuracy
0.6
0.5
0.4
0.3
0.2
0.1
0
2
4
6
8
10
12
14
16
18
Iterations
val - Base line - raw Resnet50
val - Dropout 0.2
val - Dropout 0.3
val - Dropout 0.5

Loss





Training time

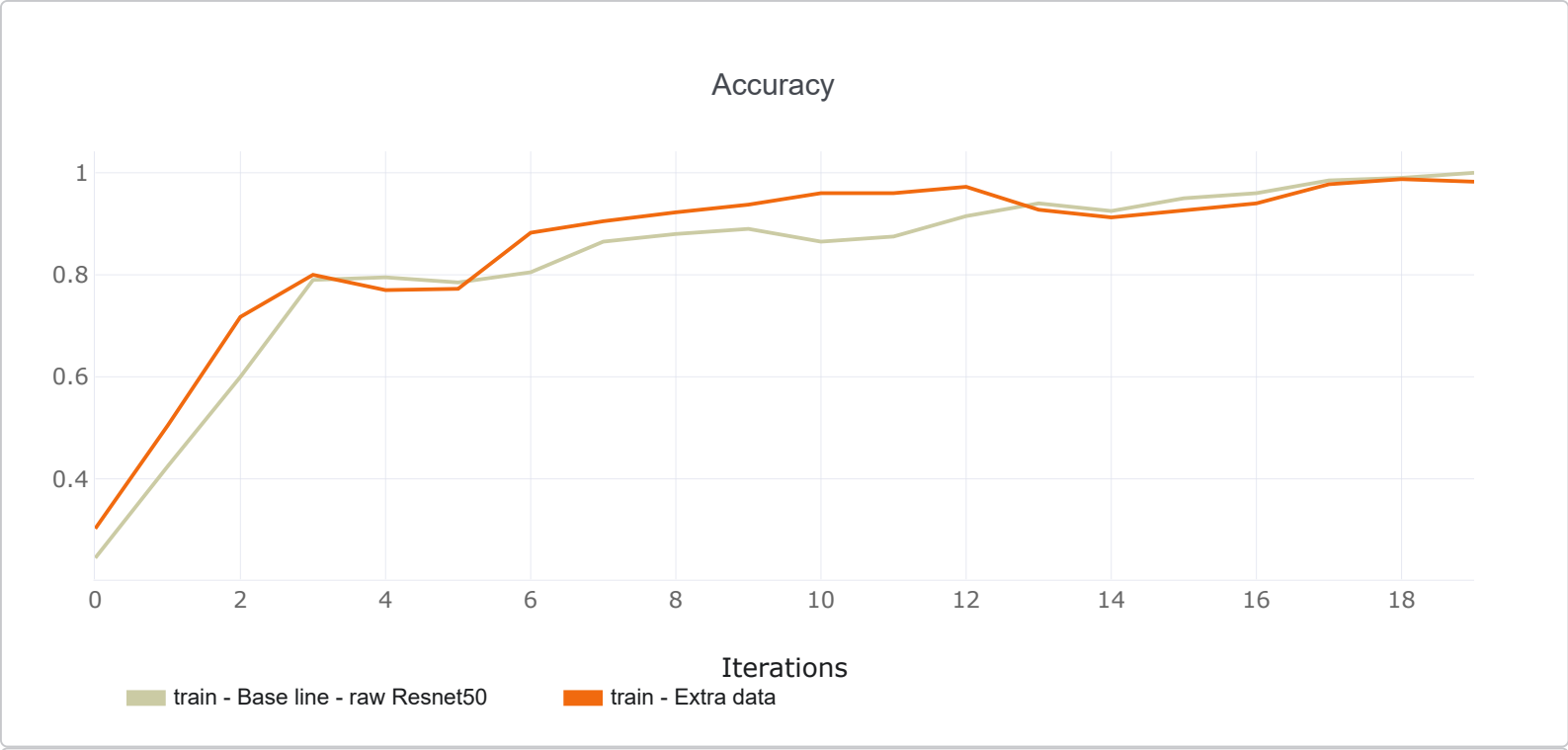


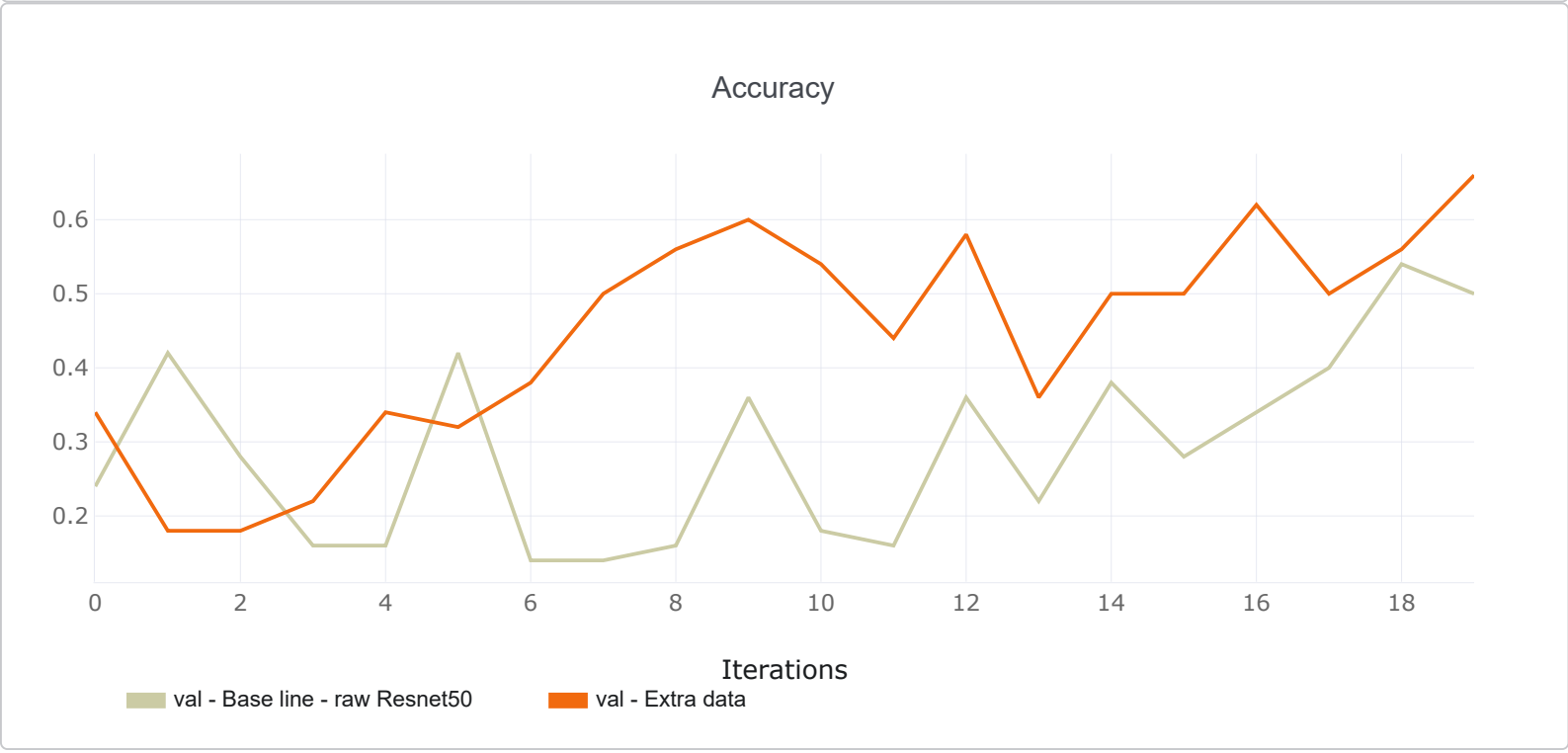
0.2 - 30% 0.3 - 52% 0.5 - 38%

Ustawienie Dropoutu na poziomie 0.5 lekko poprawiło dokładność z bazowego 34% na 52%. Pozostałe ustawienia nie miały znaczącego wpływu - 0.2 minimalnie pogorszyło wynik, a 0.5 minimalnie polepszyło. Dla dropoutu 0.5 wykresy były bardziej "poszarpane" niż w innych. Czas nie zmienił się znacząco - największa różnica względem baseline to 4 sekundy dla dropoutu 0.5.

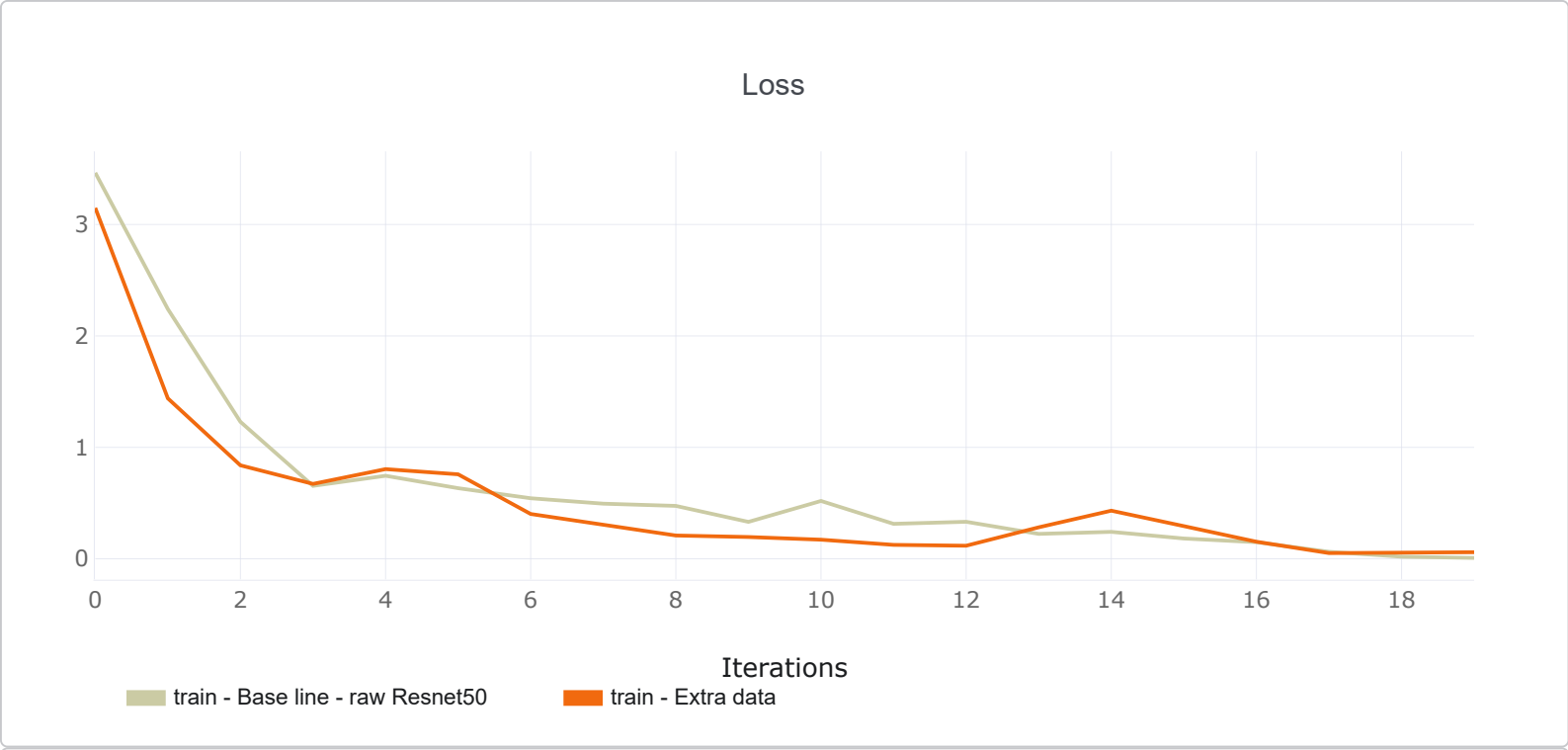
4.d - Dokładanie danych

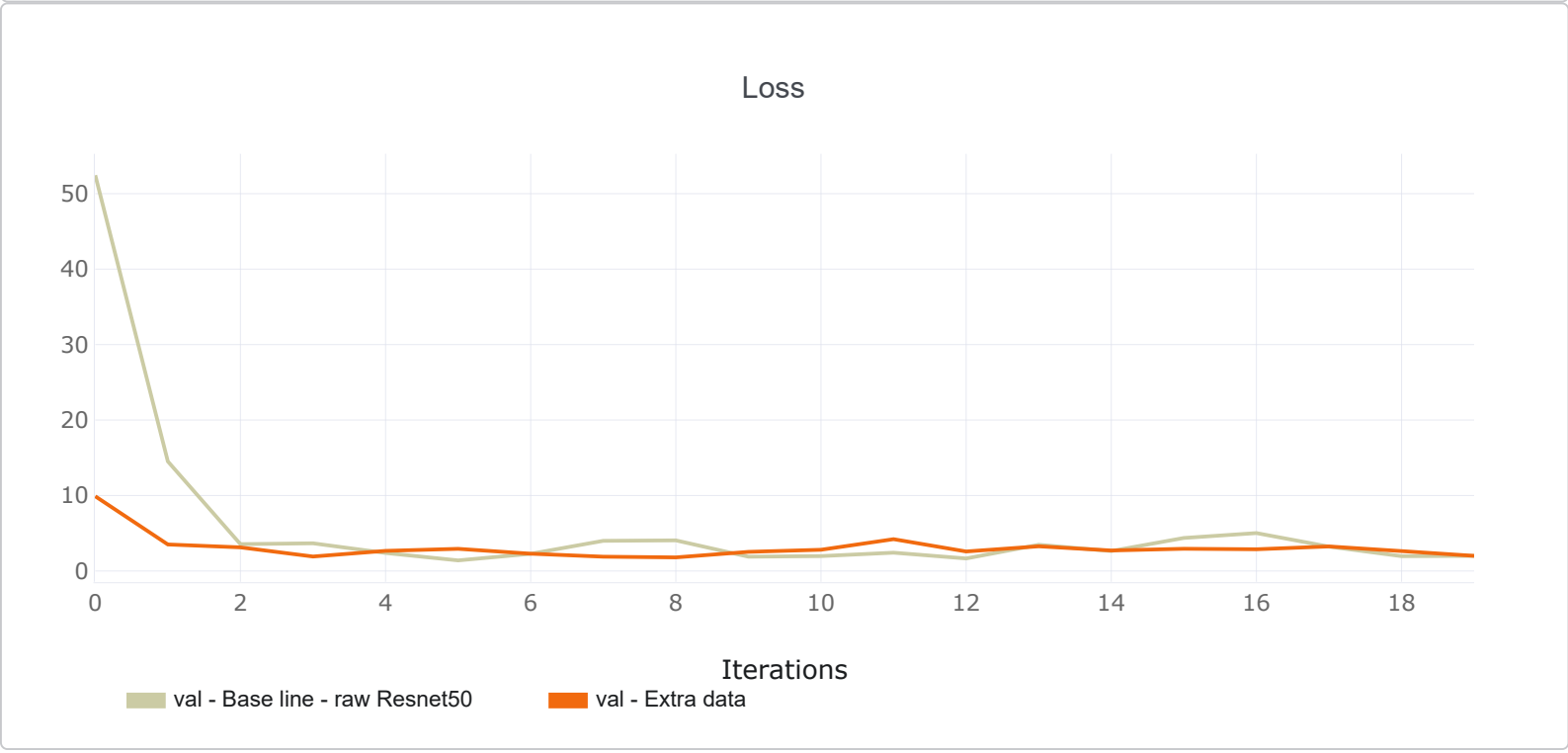
Accuracy


Accuracy
1
0.8
0.6
0.4
0
2
4
6
8
10
12
14
16
18
Iterations
train - Base line - raw Resnet50
train - Extra data


Accuracy
0.6
0.5
0.4
0.3
0.2
0
2
4
6
8
10
12
14
16
18
Iterations
val - Base line - raw Resnet50
val - Extra data

Loss


Loss
3
2
1
0
0
2
4
6
8
10
12
14
16
18
Iterations
train - Base line - raw Resnet50
train - Extra data


Loss
50
40
30
20
10
0
0
2
4
6
8
10
12
14
16
18
Iterations
val - Base line - raw Resnet50
val - Extra data

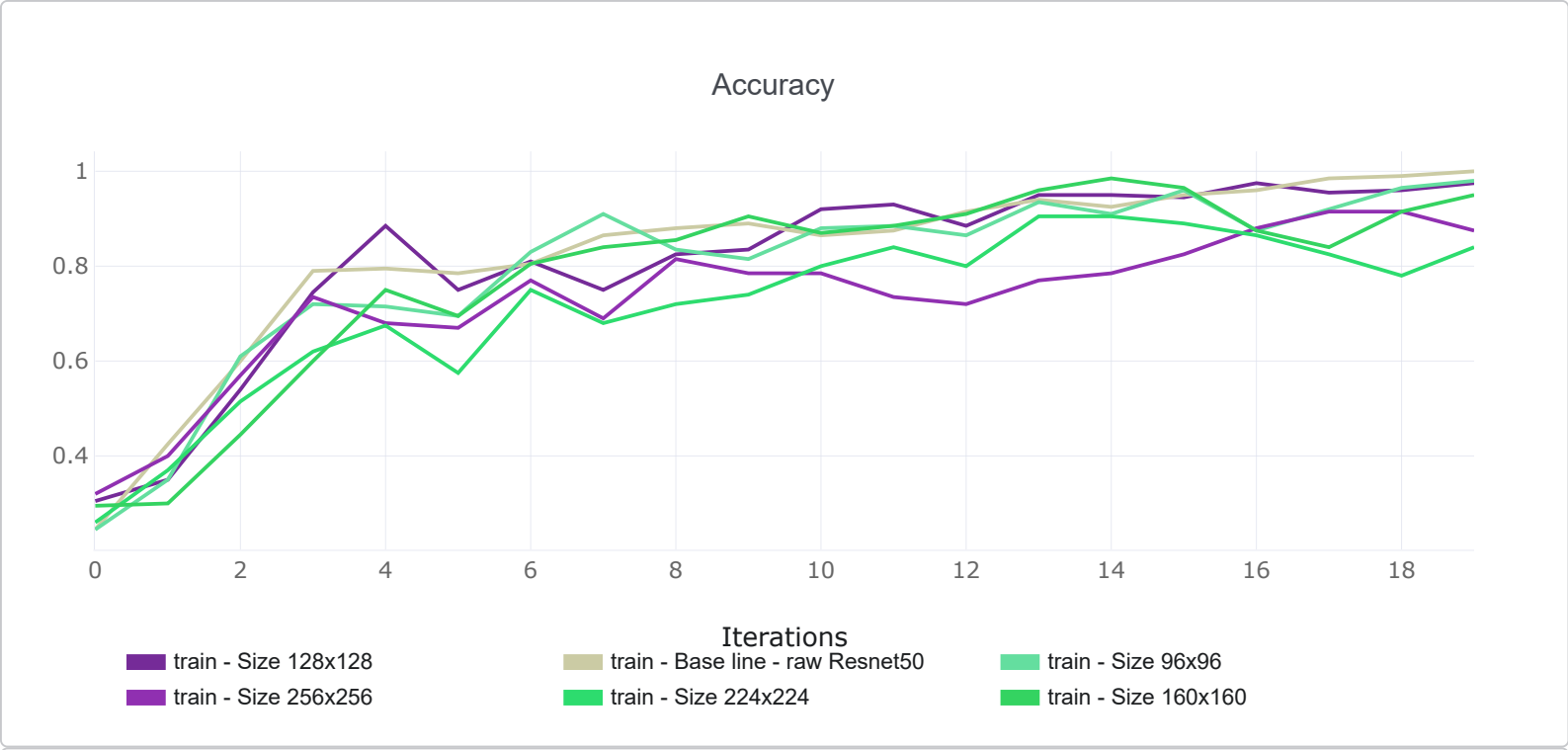
Training

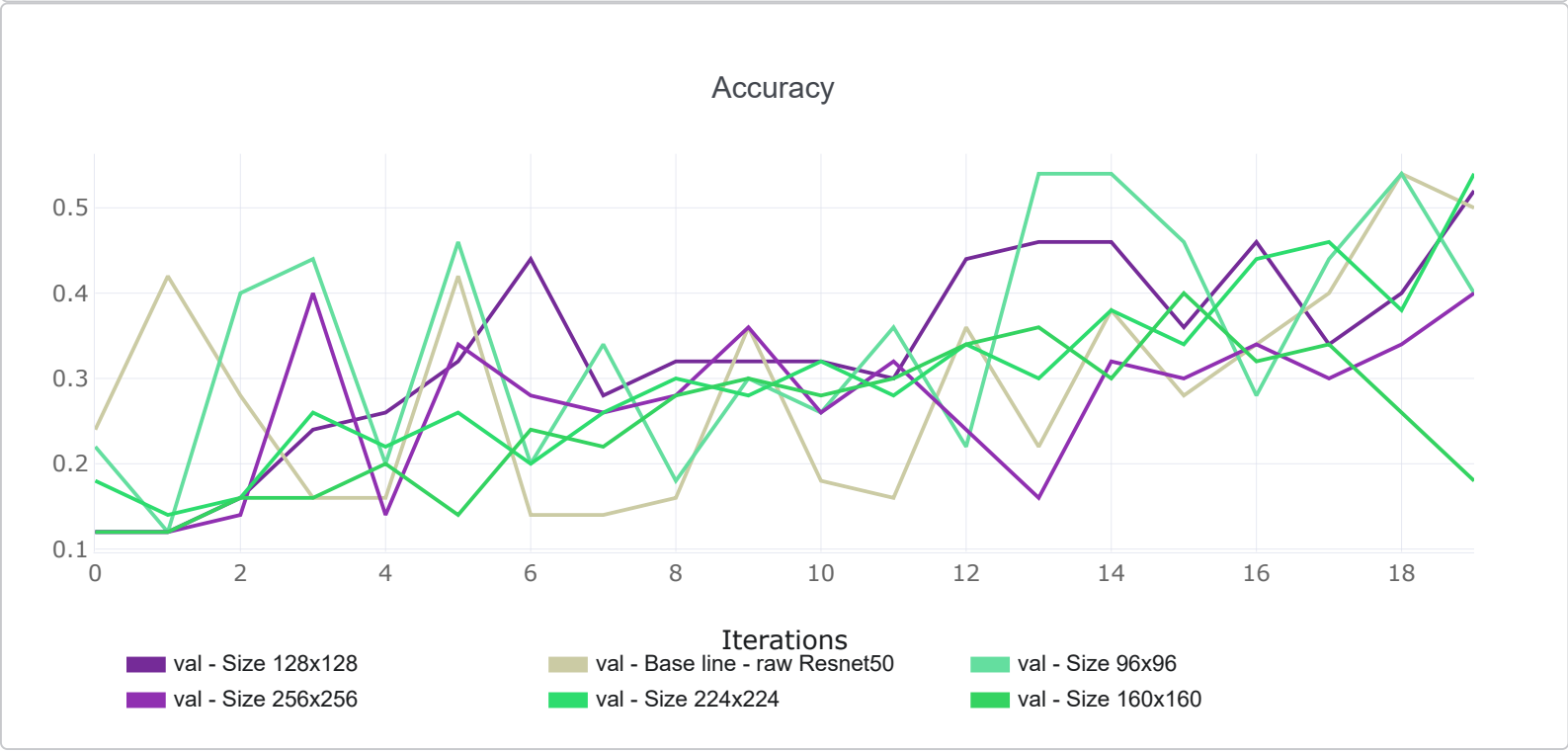


Mimo dołożenia danych za pomocją augmentacji aktualnych danych nadal wystepuje overfitting. Model zapamiętuje dane treningowe, ale nie generalizuje dobrze. Dokładnośc poprawiłą się o ~30pp, a czas uczenia wydłużył się o 100 s.

4.e - Różne rozmiary wejściowe

Accuracy


Accuracy
1
0.8
0.6
0.4
0
2
4
6
8
10
12
14
16
18
Iterations
train - Size 128x128
train - Base line - raw Resnet50
train - Size 96x96
train - Size 256x256
train - Size 224x224
train - Size 160x160

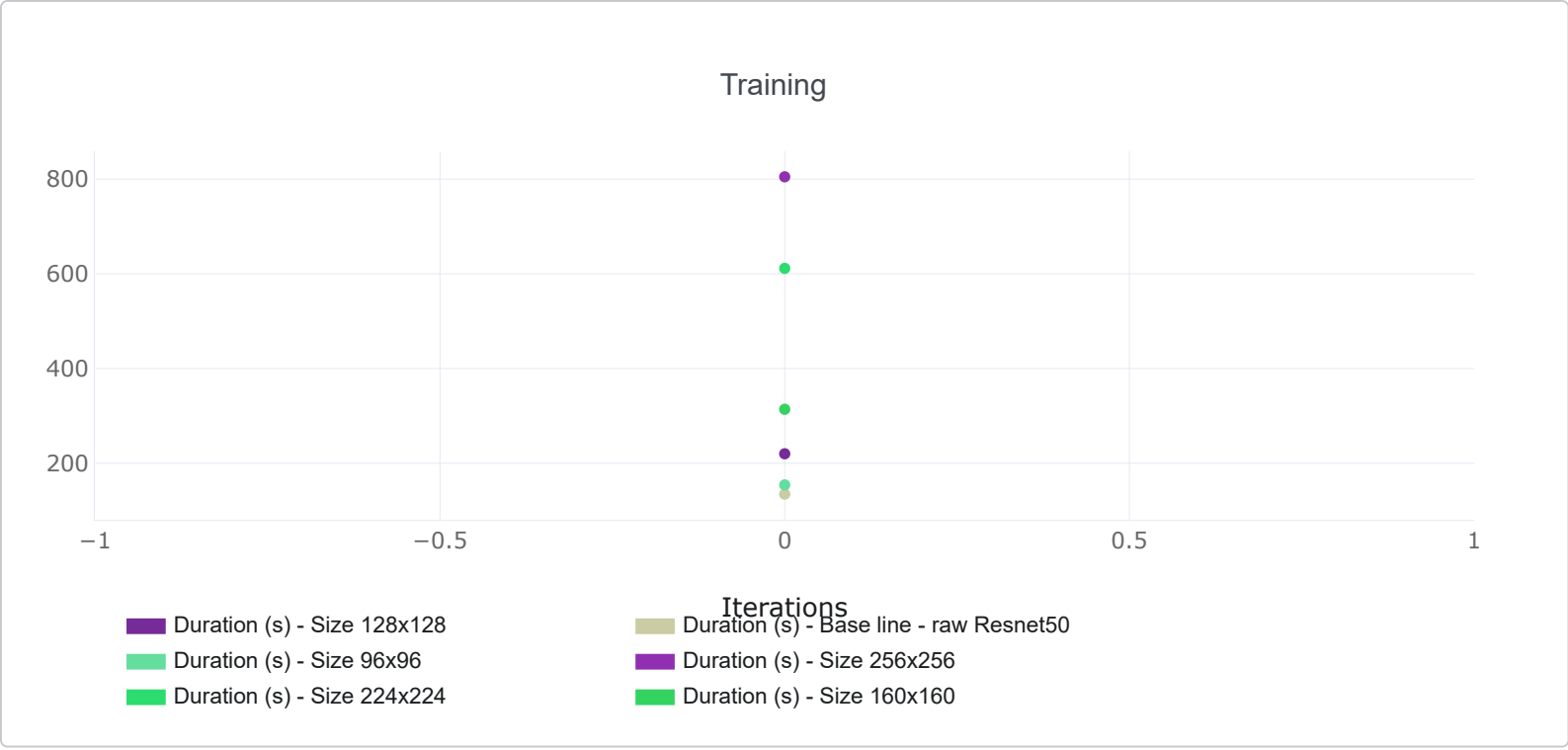

Accuracy
0.5
0.4
0.3
0.2
0.1
0
2
4
6
8
10
12
14
16
18
Iterations
val - Size 128x128
val - Base line - raw Resnet50
val - Size 96x96
val - Size 256x256
val - Size 224x224
val - Size 160x160

Loss





Training time



Dokładność uczenia dla poszczególnych rozmiarów wejściowych:

- **Baseline - 34%**
- 96x96 - 40%
- 128x128 - 52%
- 160x160 - 18%
- 224x224 - 54%
- 256x256 - 40%

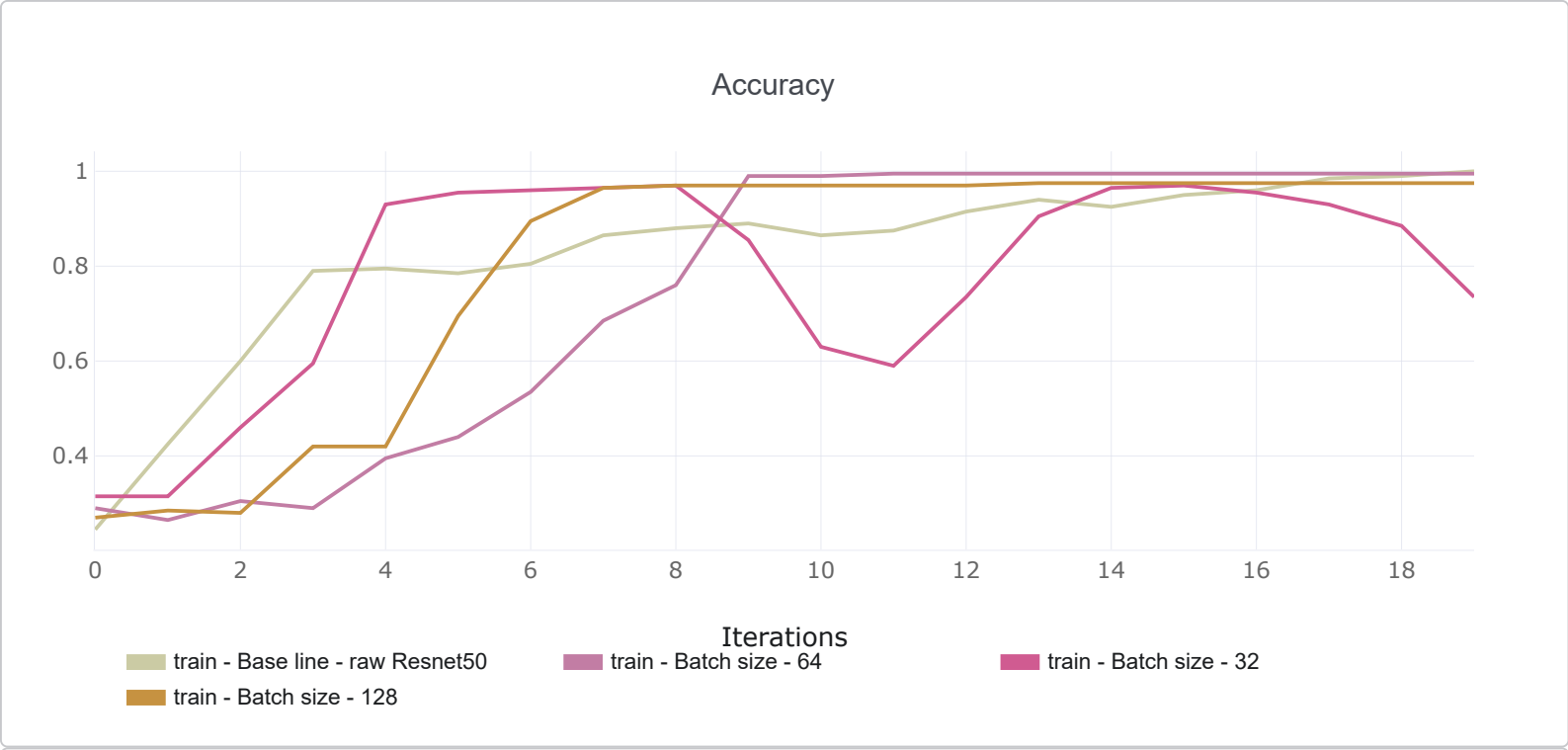
Czas uczenia:

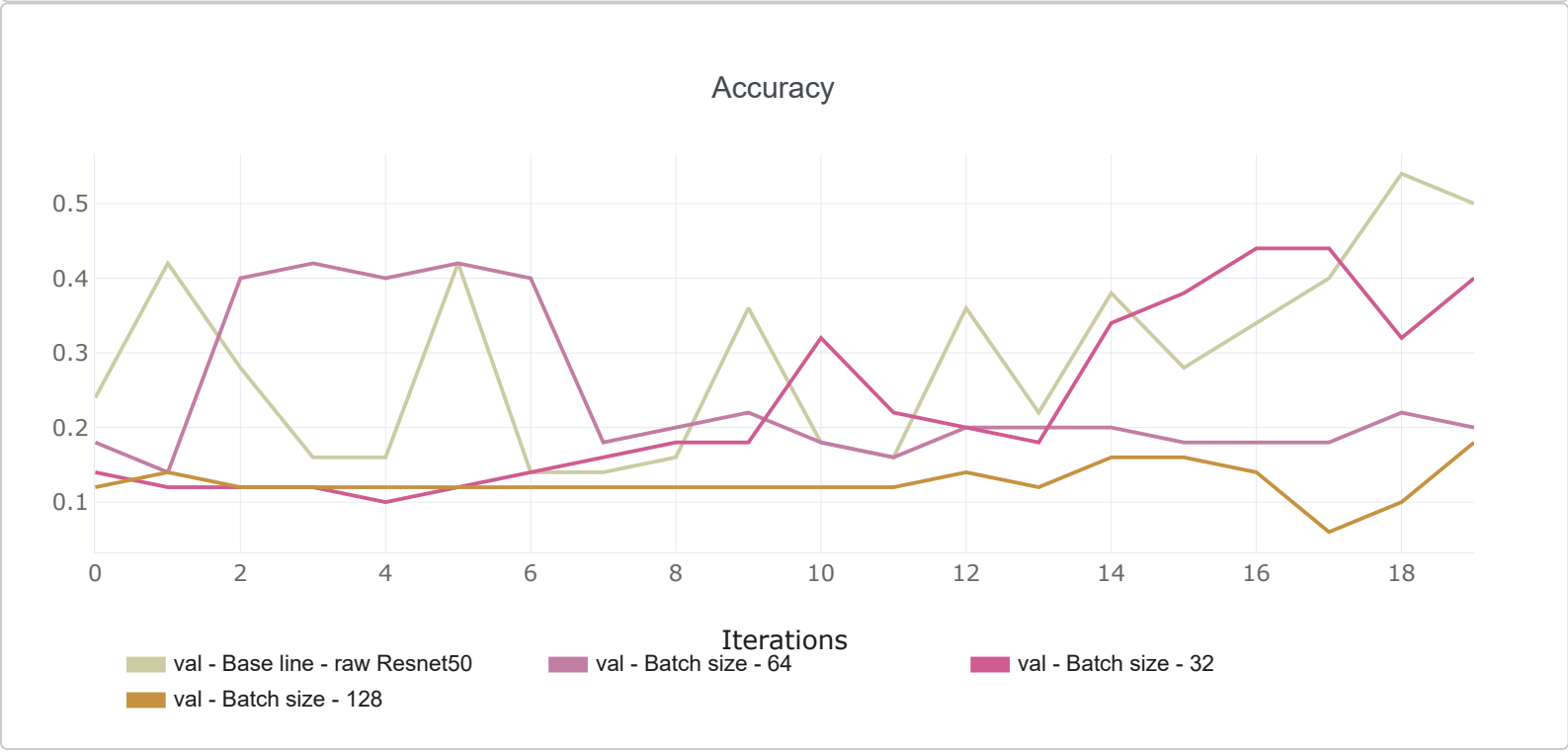
- **Baseline 64x64 - 134 sekundy**
- 96x96 - 154 sekundy
- 128x128 - 220 sekund
- 160x160 - 313 sekund
- 224x224 - 611 sekund
- 256x256 - 804 sekundy

Dla wszystkich rozmiarów poza 160×160 odnotowano poprawę względem baseline. Czas treningu rósł razem z rozmiarem zdjęć.

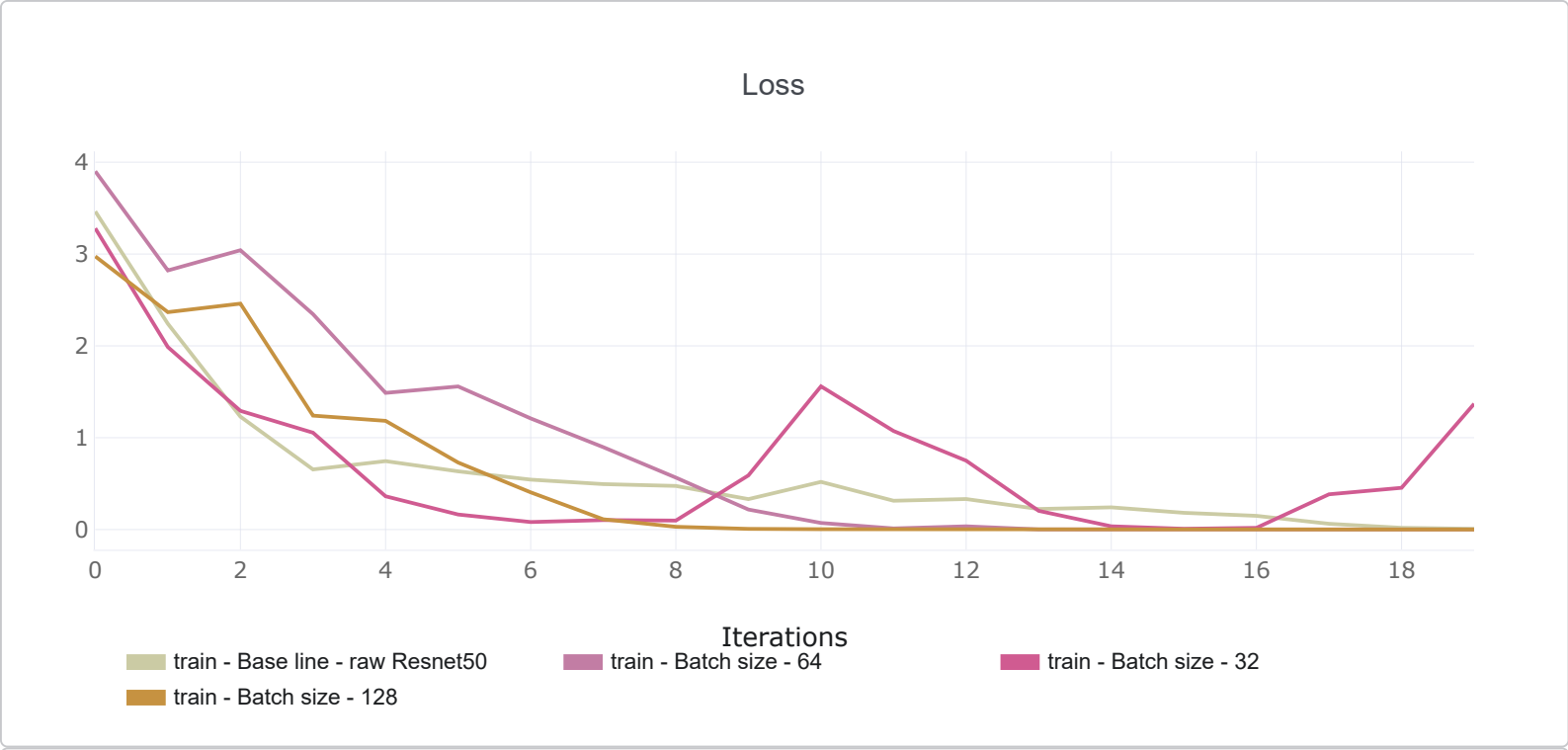
4.f - Różne rozmiary Batch size

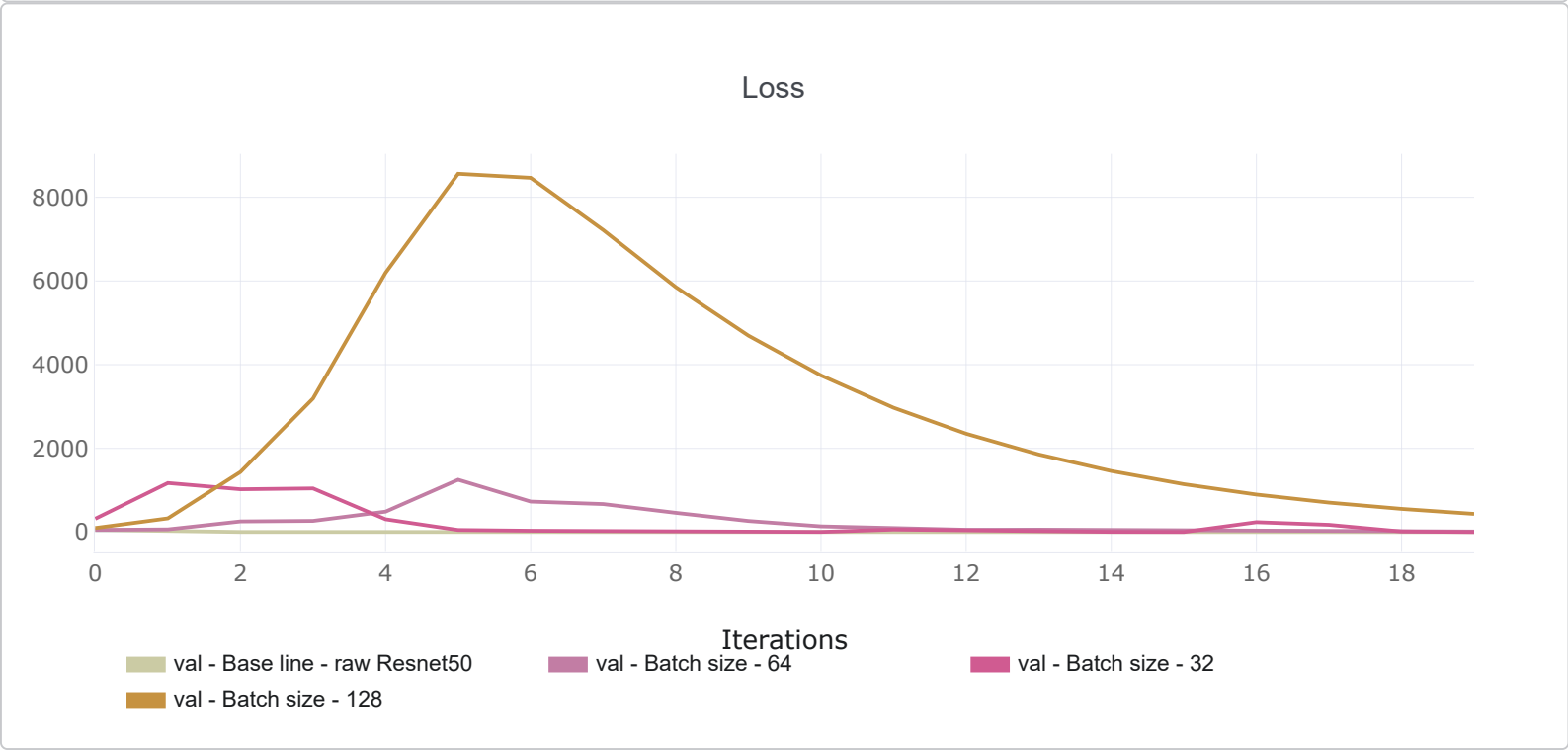
Accuracy


Accuracy
1
0.8
0.6
0.4
0
2
4
6
8
10
12
14
16
18
Iterations
train - Base line - raw Resnet50
train - Batch size - 64
train - Batch size - 32
train - Batch size - 128

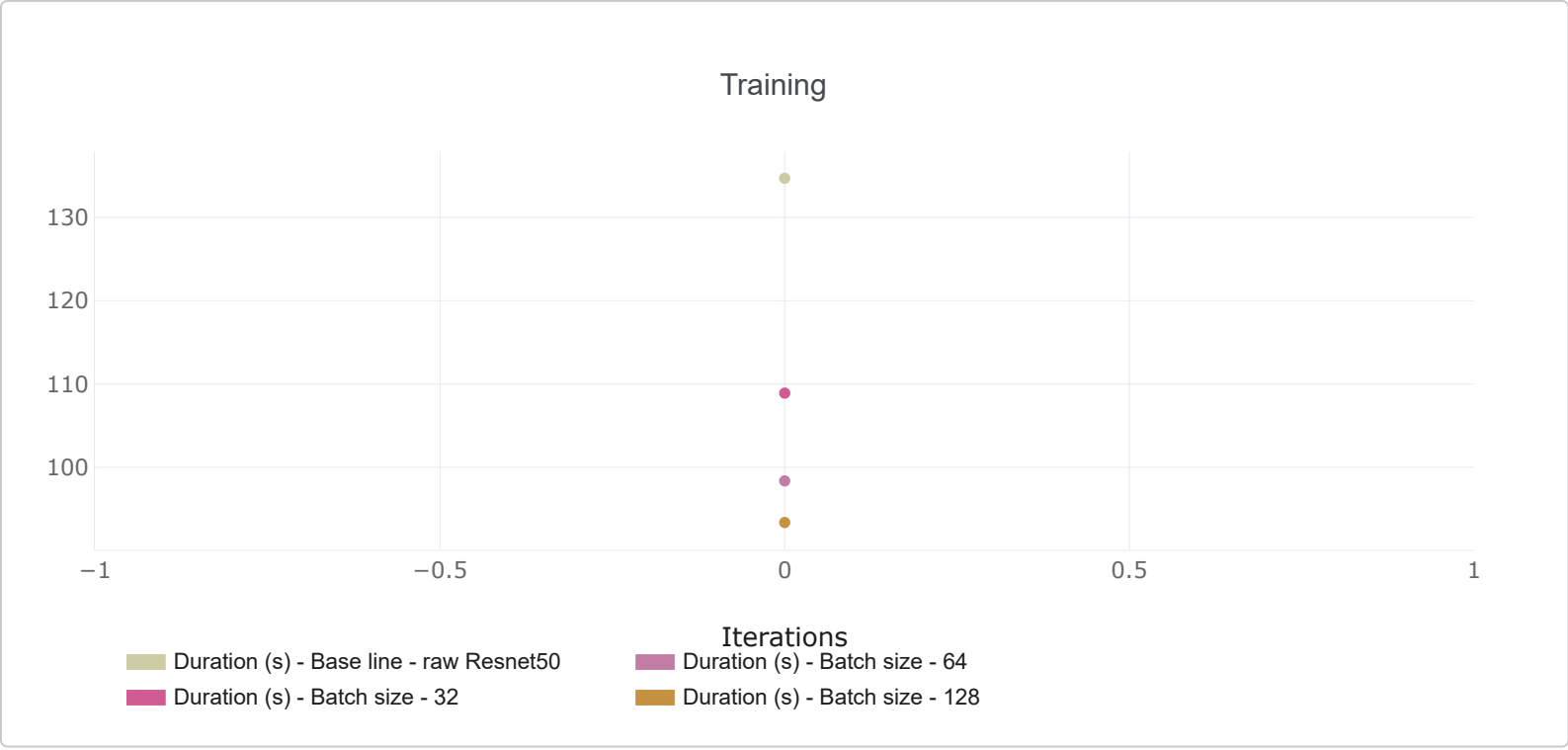

Accuracy
0.5
0.4
0.3
0.2
0.1
0
2
4
6
8
10
12
14
16
18
Iterations
val - Base line - raw Resnet50
val - Batch size - 64
val - Batch size - 32
val - Batch size - 128

Loss





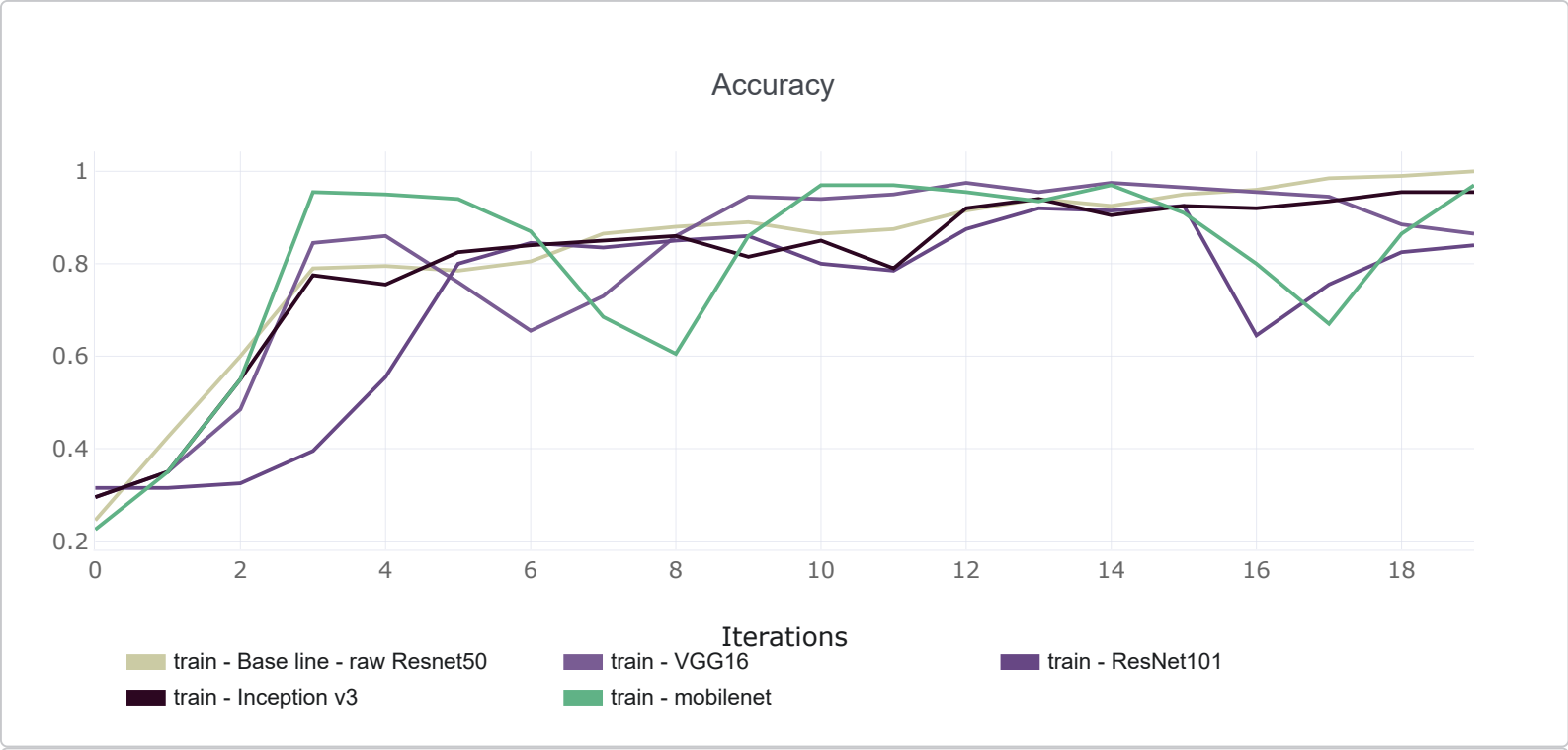
Training time

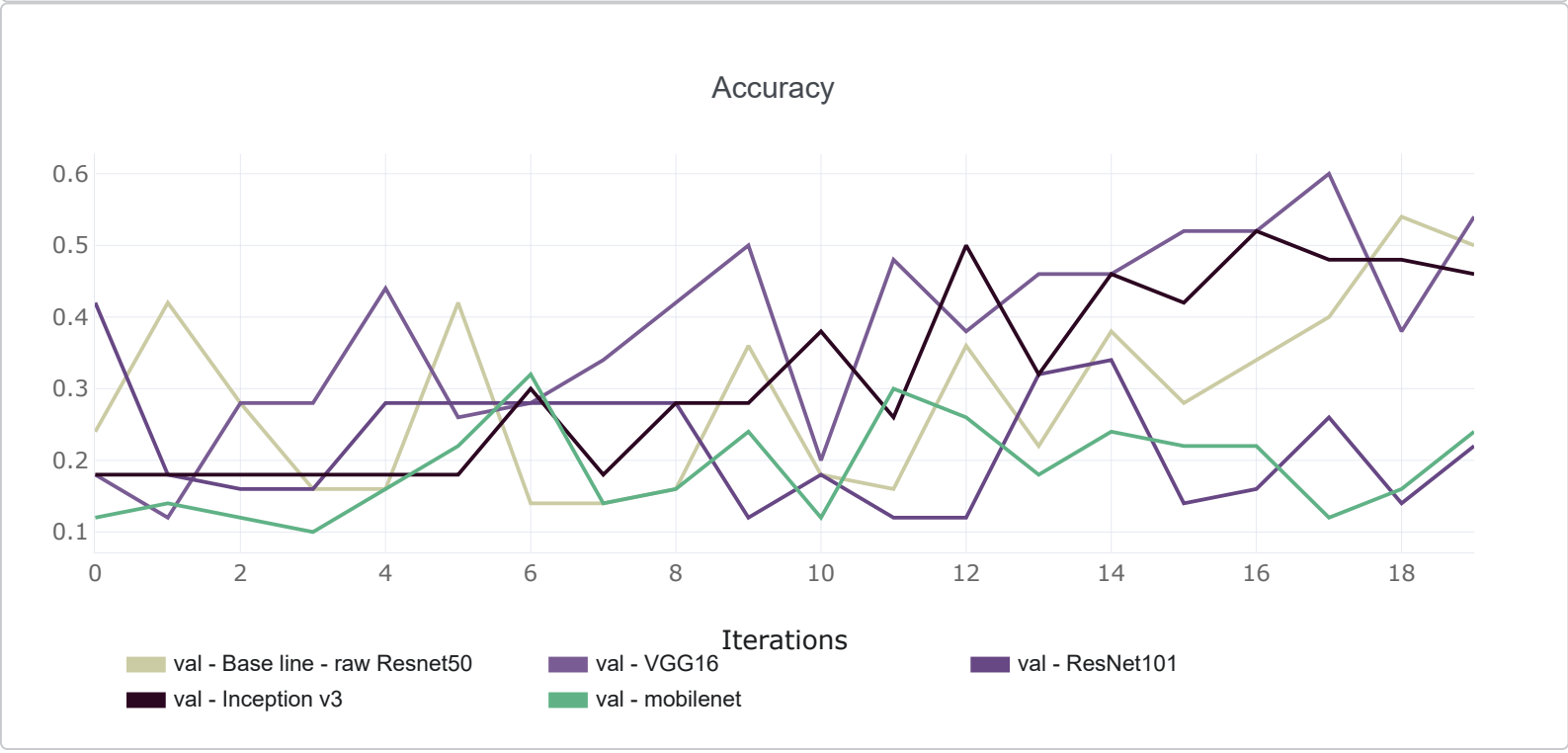


Wraz ze zwiększaniem batch size model coraz szybciej się przeuczał jak. Analogicznie czas uczenia się skracał.

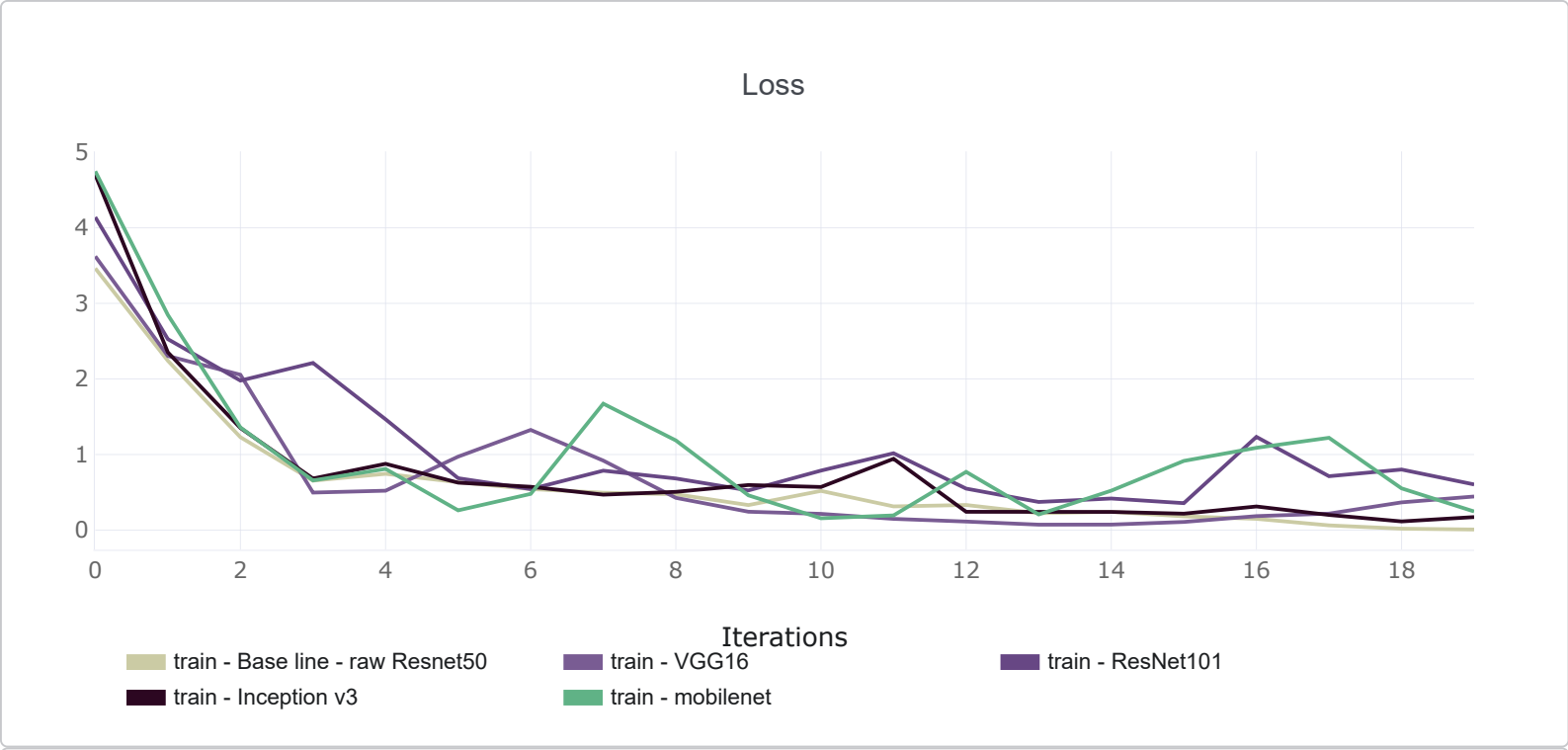
4.g - Różne struktury sieci

Accuracy


Accuracy
1
0.8
0.6
0.4
0.2
0
2
4
6
8
10
12
14
16
18
Iterations
train - Base line - raw Resnet50
train - VGG16
train - ResNet101
train - Inception v3
train - mobilenet

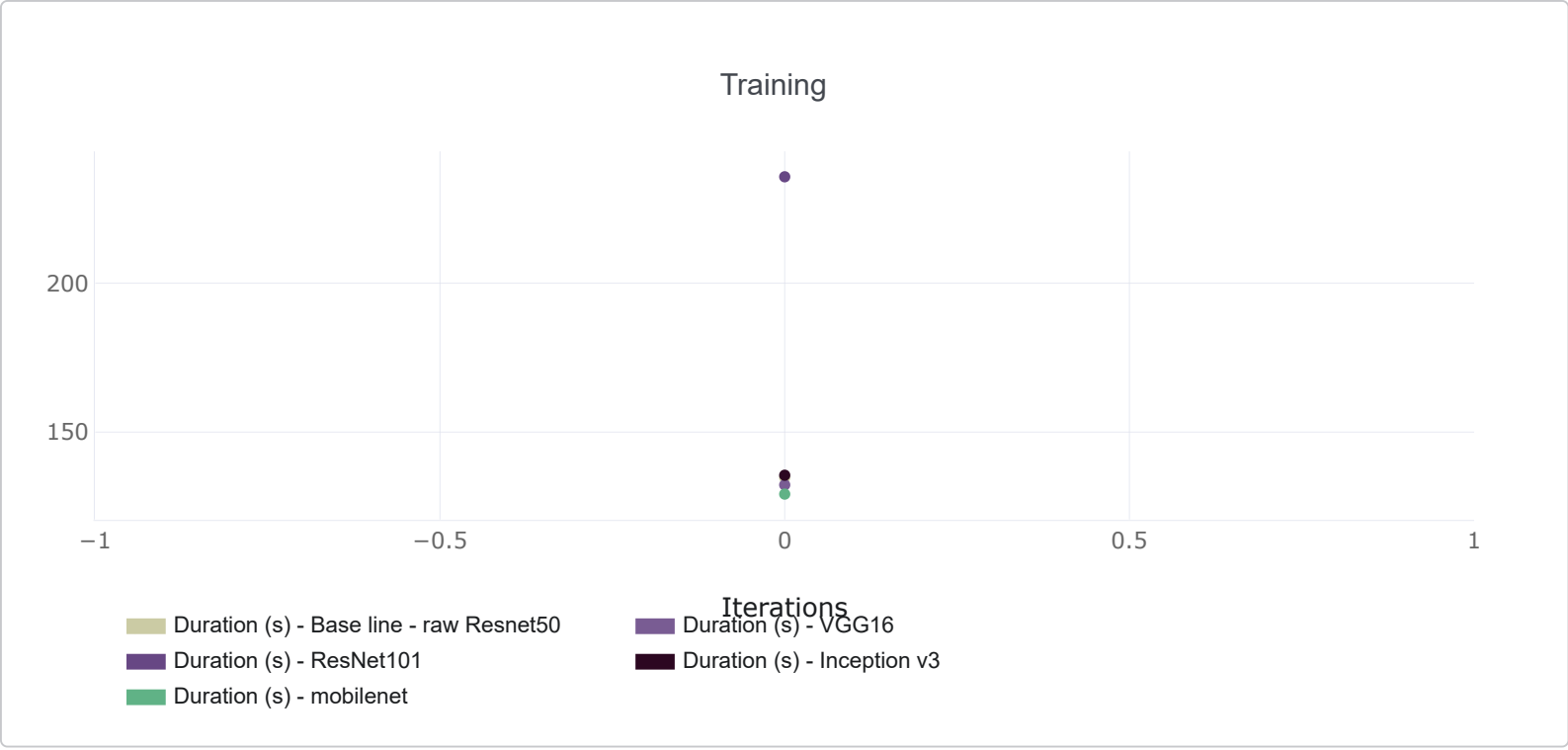

Accuracy
0.6
0.5
0.4
0.3
0.2
0.1
0
2
4
6
8
10
12
14
16
18
Iterations
val - Base line - raw Resnet50
val - VGG16
val - ResNet101
val - Inception v3
val - mobilenet

Loss





Training time



Dokładność poszczególnych modeli:

- **Baseline - ResNet50 - 34%**
- ResNet101 - 22%
- VGG16 - 54%
- Inception v3 - 46%
- mobilenet - 24%

Modele VGG16 i Inception v3 poprawiły bazowy wynik ResNet50. ResNet101 i mobilenet działały z dokładnością mniejszą o ok. 10%. Czas uczenia wszystkich modeli jest bardzo podobny do mdelu bazowego z wyjątkiem ResNet101, który prawie podwoił czas działania modelu bazowego.

Wnioski:

Baseline - "surowy" ResNet-50

- 64×64, CPU, bez transfer-learningu, bez żadnych technik regularyzacji → val accuracy = 34 % przy 134 s czasu uczenia.
- Już po 3-4 epokach rozjazd między acc-train a acc-val ujawnił silne przeuczenie – model zapamiętuje 200 obrazków treningowych, ale nie generalizuje.

Optymalizacja szybkości

- GPU (T4) - val accuracy = 64%, czas treningu = 128s - 6s -> Dwukrotna poprawa trafności mimo symbolicznego zysku czasowego (mały model + mały zbiór)
- Transfer learning - val acc = 80%, czas treningu = 27s -> Największy skok jakości (+46 pp) i najszybszy czas; potwierdza, że przy małych zbiorach nie opłaca się trenować od zera

Optymalizacja dokładności

- Normalizacja - val acc 48% - lekka poprawa ale sama nie wystarcza
- Augmentacja - val acc 62% - wyraźnie redukuje overfitting
- Dropout 0.3/0.5 - val acc 52/38% -0.3 stabilizuje, 0.5 już zbyt agresywny
- Dokładanie danych - val acc 64% - mimo lepszej ogólnej wydajności, model wciąż ma trudności z rozpoznawaniem niektórych klas

Wpływ hiperparametrów

- **Rozmiar obrazu** - 128×128 i 224×224 podniosły trafność do ≈ 52-54 %, ale kosztem 2-4× dłuższego uczenia. 128×128 okazało się najlepszym kompromisem.
- **Batch size** - większe batch-size (64, 128) przyspieszały uczenie, lecz silnie zwiększały przeuczenie; 16 i 32 daje najlepszy balans przy tym zbiorze.
- **Modele** - VGG-16 (54 %) i InceptionV3 (46 %) przewyższyły surowy ResNet-50; ResNet-101 i MobileNet spadły < 30 %. VGG-16 jest dobrą alternatywą, jeśli zależy nam na prostej sieci bez TL.

5. Opytmalizcja hiperparametrów

Opytmalizacja została przeprowadzona kilka razy ale hiperparametry wybrane jako najlepsze w tym procesie dawały bardzo słabe wyniki. Tutaj przykładowe logi optymalizacji w której wykonano 58 eksperymentów:

```
Progress report #1 completed, sleeping for 0.5 minutes..

Progress report #58 completed, sleeping for 0.034163808822631835 minutes

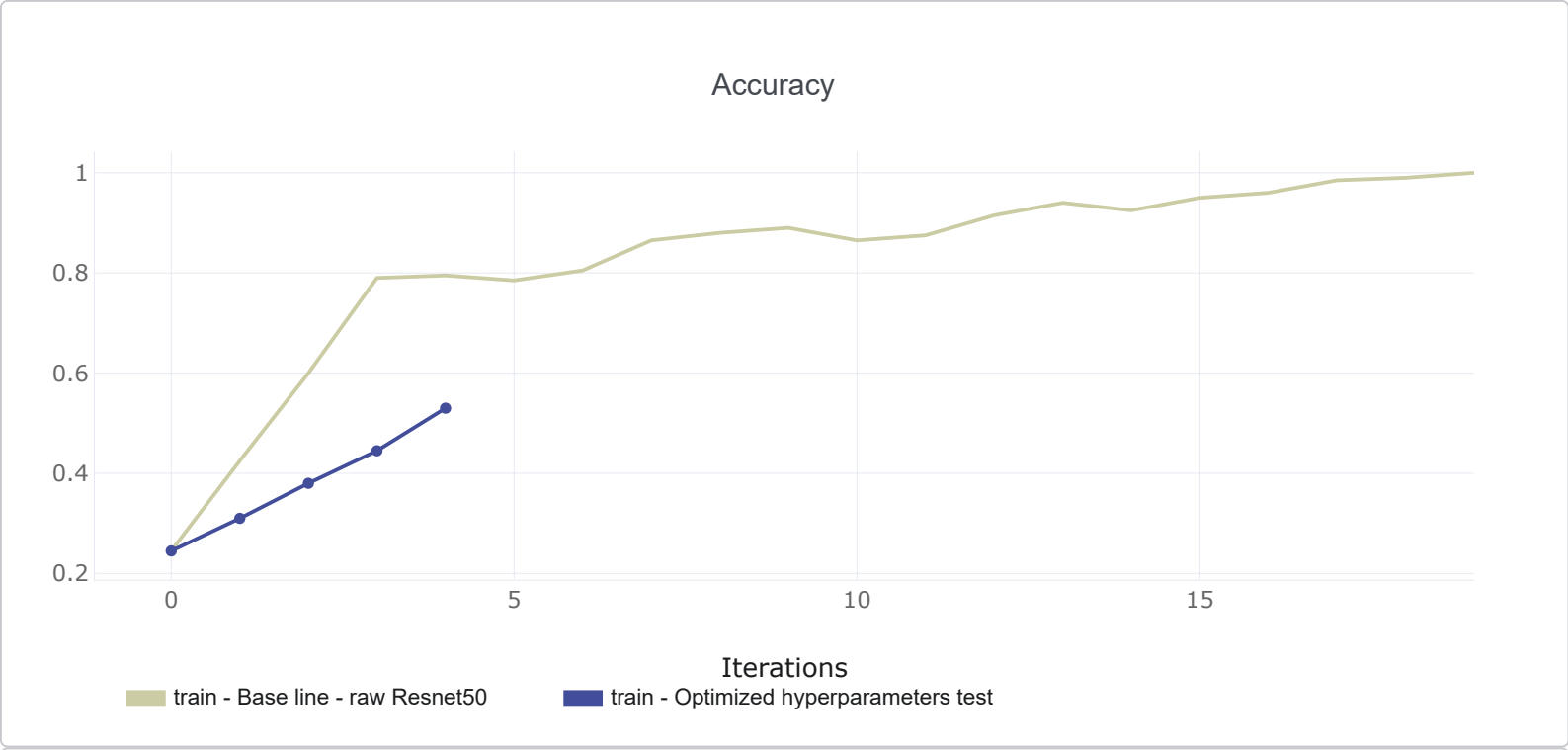
=== Najlepszy eksperyment ===

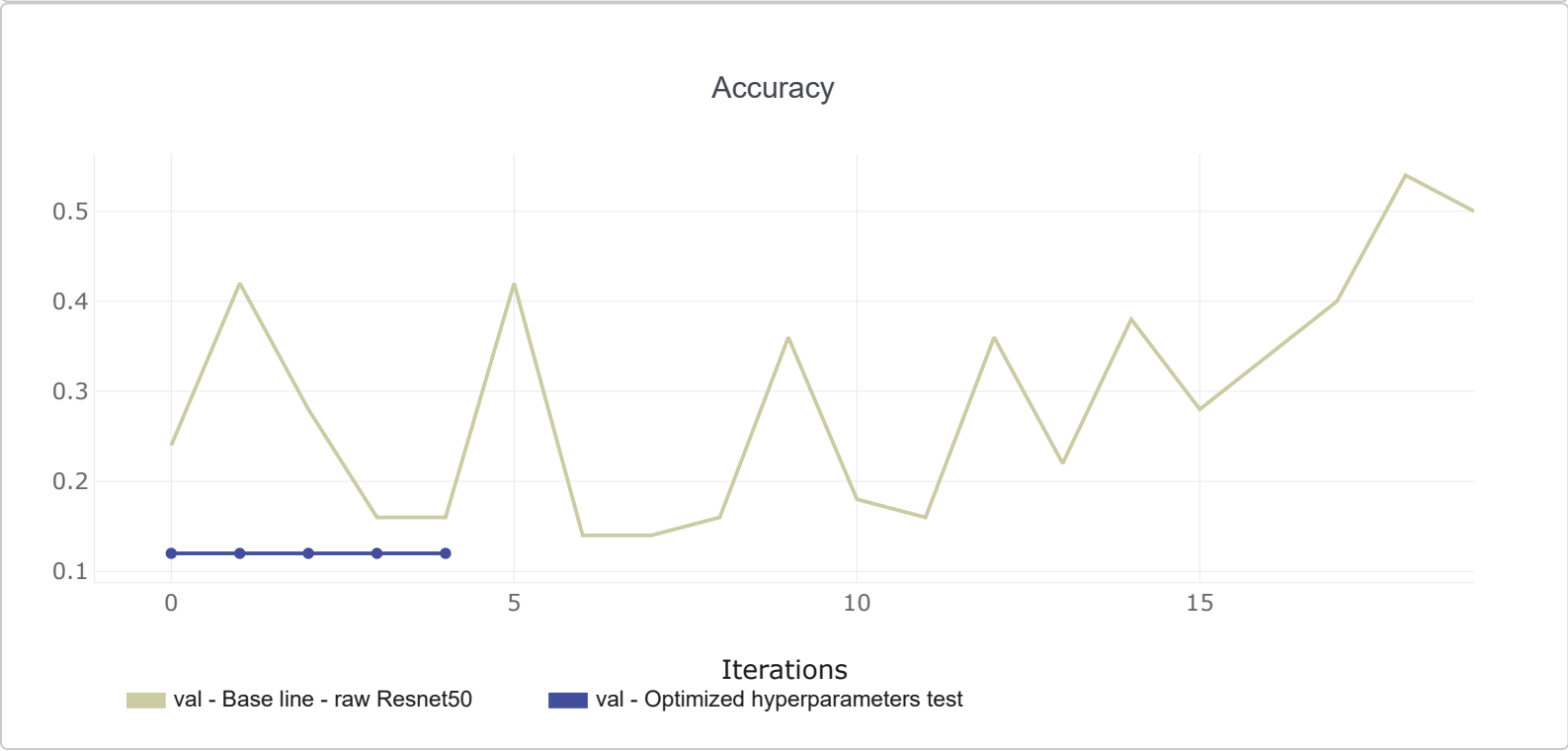
Task ID: 9ff59154189c4d93aa2e34e08746838d

Hiperparametry:
  General/model_name: mobilenet
  General/img_shape: 128.0
  General/batch_size: 48.0
  General/transfer_learning: False
  General/normalization: False
  General/augmentation: False
  General/dropout: 0.4
  General/extra_data: False
  General/epochs: 5.0
```

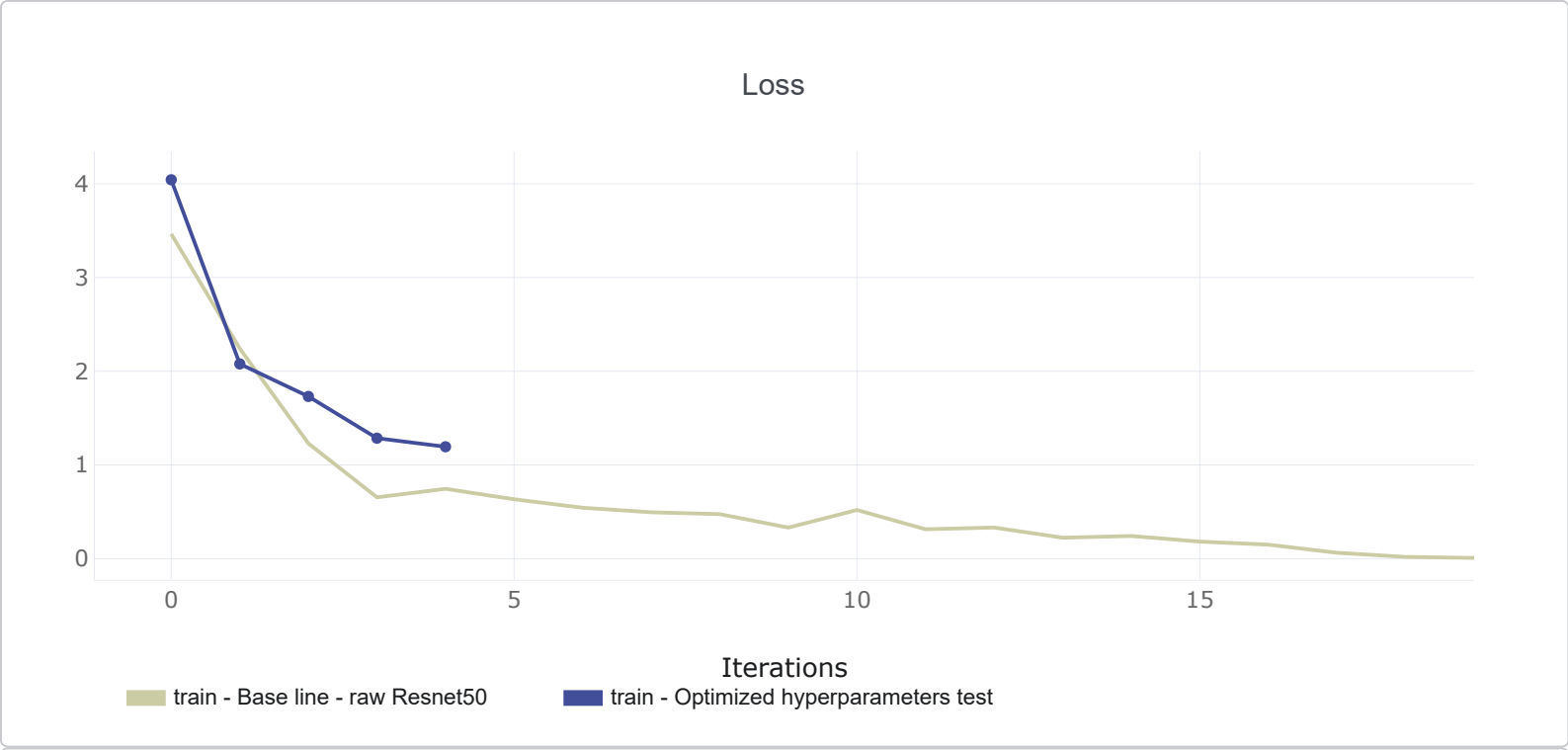
Porównanie bazowego wyniku z wynikiem na zoptymalizowanych automatycznie hiperparametrach

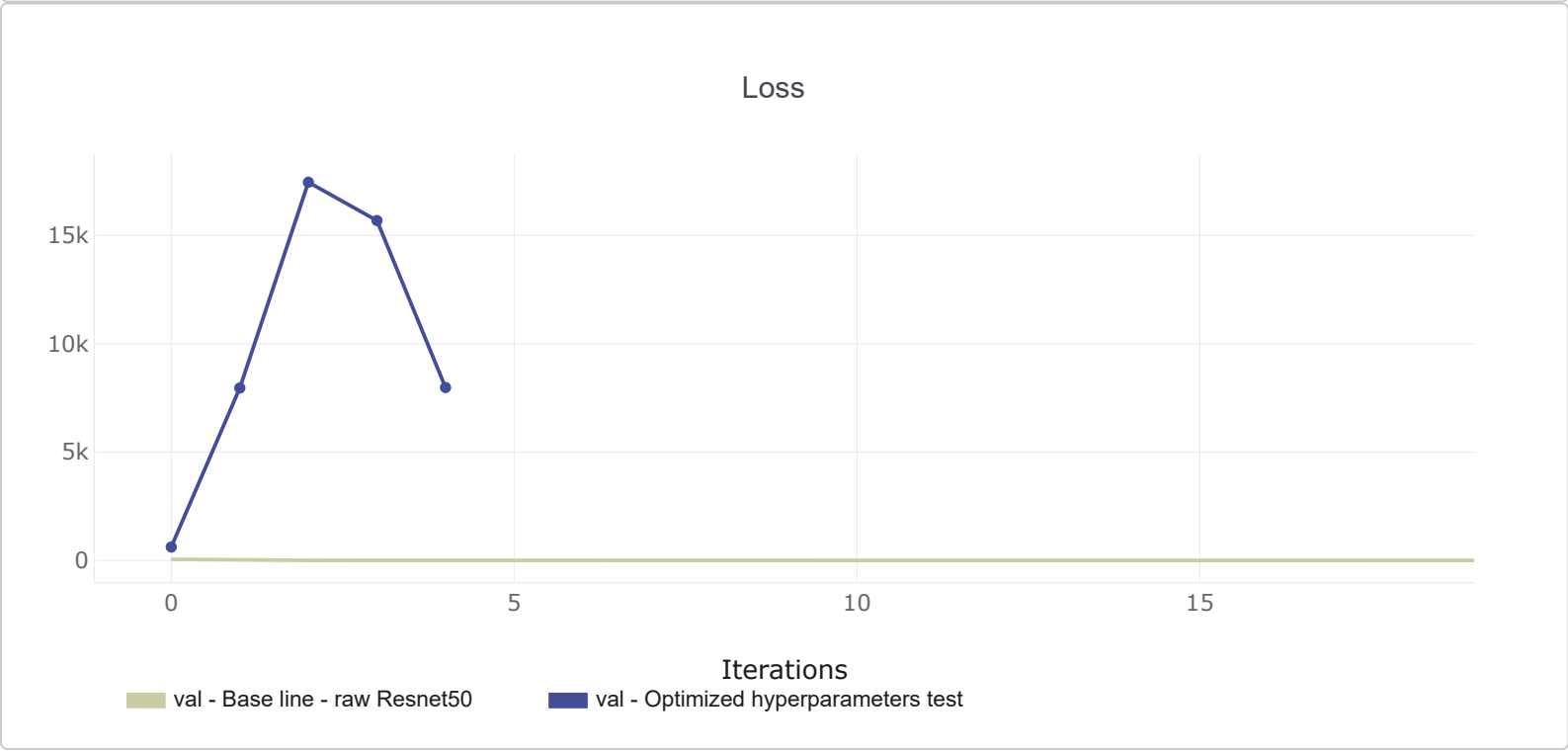
Accuracy


Accuracy
1
0.8
0.6
0.4
0.2
0
5
10
15
Iterations
train - Base line - raw Resnet50
train - Optimized hyperparameters test

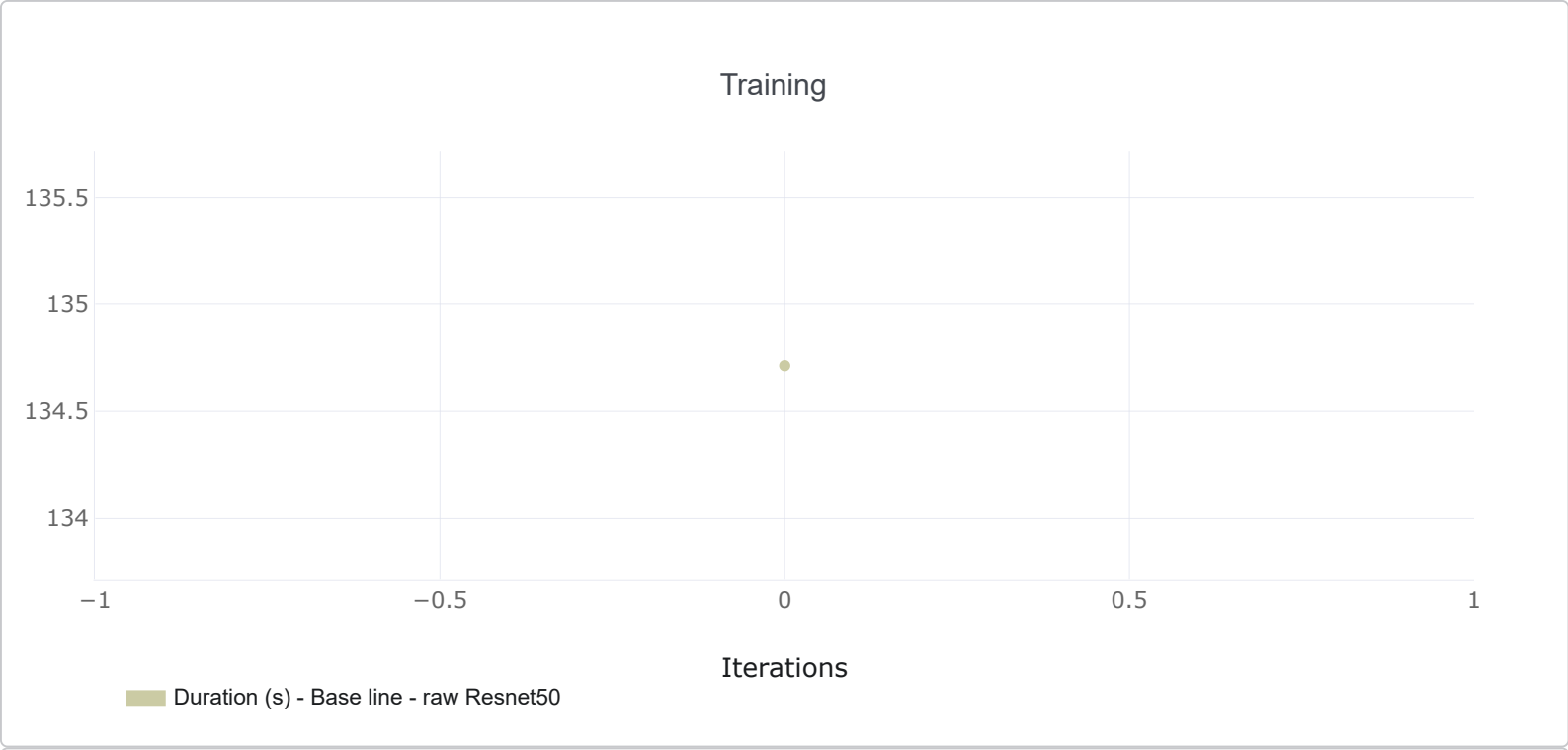

Accuracy
0.5
0.4
0.3
0.2
0.1
0
5
10
15
Iterations
val - Base line - raw Resnet50
val - Optimized hyperparameters test

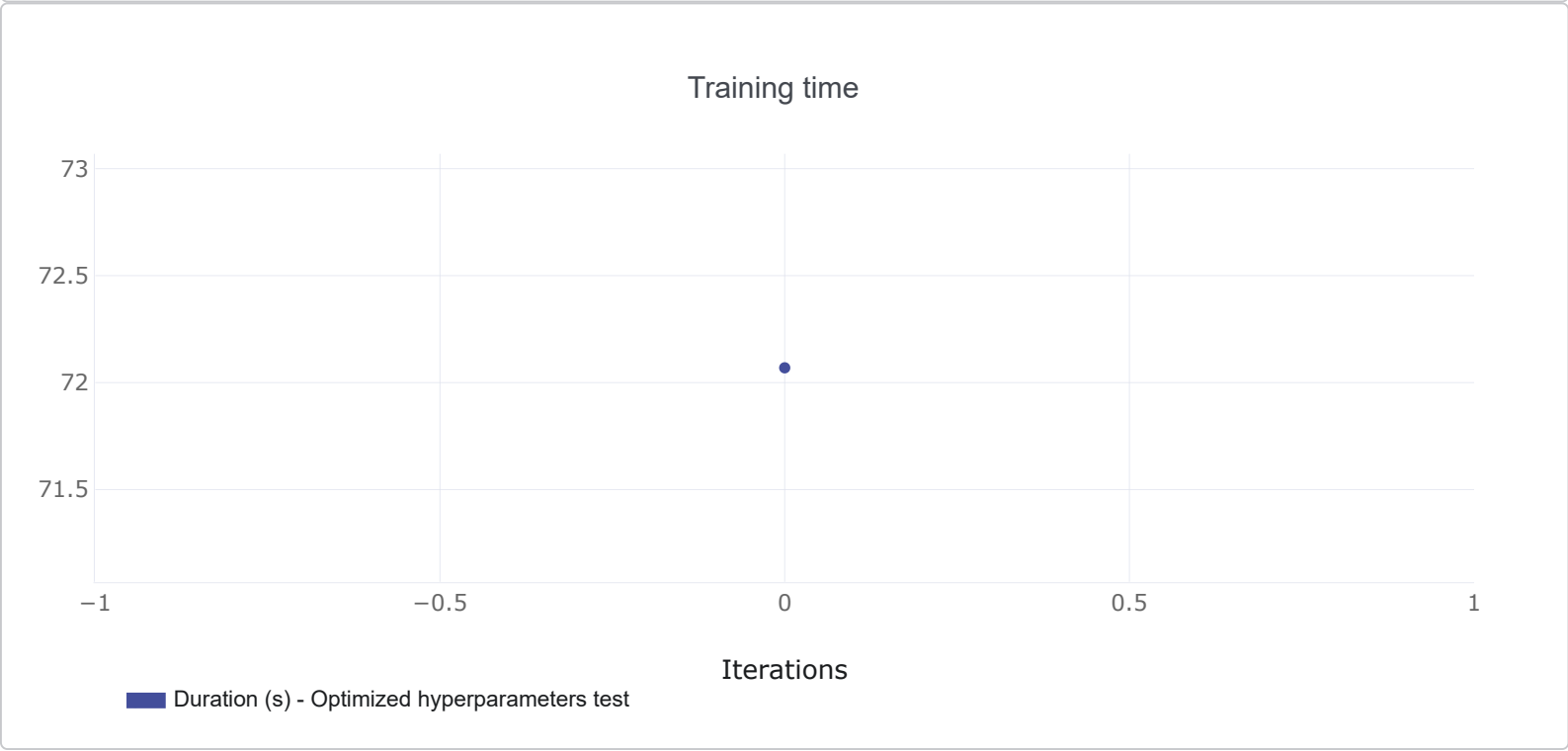
Loss





Training time


Training
135.5
135
134.5
134
−1
−0.5
0
0.5
1
Iterations
Duration (s) - Base line - raw Resnet50


Training time
73
72.5
72
71.5
−1
−0.5
0
0.5
1
Iterations
Duration (s) - Optimized hyperparameters test